AURA: Angular Update Rate Adaptation for training complex-valued neural networks

Enrico Ballini[1], Allan Peter Engsig-Karup[2], and Tito Andriollo[1]

arXiv:2609.25914v1 [cs.LG] 22 Sep 2026

***Abstract*—Complex-valued neural networks (CVNNs) are increasingly adopted for complex-valued data; however, they are often trained with first-order optimizers inherited from the real-valued case. The efficiency of these methods depends largely on the step size, and their step-size rules ignore the angular information available in the complex plane. We address step-size adaptation in the complex domain by introducing AURA (Angular Update Rate Adaptation), a per-parameter step-size adaptation that can be added on top of any first-order optimizer, and removed from it, without altering its update direction. AURA measures the agreement between consecutive updates of each complex parameter, in length, alignment, and sense of rotation, and enlarges the step when they are consistent and reduces it when they are not. It requires no additional gradient evaluations and only inexpensive vector operations per step. We combine AURA with Adam and Muon and compare the resulting methods with well-known first-order optimizers on four test cases of increasing complexity, ranging from the approximation of scalar complex functions to physics-informed training. Fully connected neural networks are used throughout this work. All hyperparameters other than the step size are held fixed across test cases; for one case, we also tune the hyperparameters of each optimizer under the same budget. Our empirical tests show that AURA improves the convergence of its base optimizer in most cases with a small per-step overhead, and we identify the conditions under which it fails to do so.**

***Index Terms*—Complex-valued neural networks, adaptive optimization, step-size adaptation**

I. Introduction

Neural networks are trained with iterative optimizers, ranging from first- to second-order methods and from heuristic rules to principled ones [1]. First-order methods are the standard in deep learning because each step is cheap, but their efficiency depends largely on the step size. The classical levers are the gradient history and its reliability: momentum lets noise cancel and consistent directions accumulate, while the signal-to-noise ratio (SNR) of the gradient decides how far a step can be trusted, as in AdaBelief [2].

A further device to adjust the step size is the agreement between consecutive gradients or updates; for real-valued networks, AngularGrad [3] rescales the Adam step with an angle-like measure of consecutive gradients, diffGrad [4] with their difference, and AdaSmooth [5] sets the averaging window of the RMSProp denominator from the ratio between the net and the total displacement of a weight. Hypergradient descent [6], applied earlier to complex-valued adaptive filters [7], [8], learns the step size itself by gradient descent, and its multiplicative form updates the step size with the cosine between the current gradient and the previous update. Adaptive methods such as AdaGrad [9] and AdaDelta [10] keep one step size per parameter, and CoRe [11], [12] shows that adding an RPROP-like sign test to Adam at this level can be effective in real-valued networks.

Whether such rules actually speed up training can only be judged through fair comparisons. Benchmarking practices are set in, e.g., [13] and [14], based on standard test problems and explicit tuning budgets, and take into account that the hyperparameter search space can change the ranking [15].

The above works focused on the real-valued case, see also [16]. For complex-valued neural networks (CVNNs), [17] compare adaptive optimizers in online training only, and [18] show that the specific optimizer changes how CVNNs compare with real-valued neural networks (RVNNs).

CVNNs process complex-valued data directly and are used in image classification [19], image reconstruction, signal classification, speech enhancement, wind prediction, and control systems [20]. They are a natural choice in telecommunications [17], and in MRI reconstruction they outperform real-valued CNNs with the same number of parameters [21]. Their advantage is not universal, however: [22] find RVNNs as good or better in most cases, and the optimizer is part of the reason [18].

Several training algorithms have been designed for CVNNs, from metacognitive learning in complex RBF networks [23] to conjugate gradients and selectable search directions with Armijo [24] and Wolfe [25] line searches. Adaptive optimizers, instead, have been ported from the real-valued case by treating the real and imaginary parts separately: [26] extend Adam in this way, and [17] extend AdaGrad [9], RMSProp [27], AdaMax [28], AMSGrad [29], SAMSGrad [30], Nadam [31], and diffGrad [4]. Their step-size rules therefore act on each part independently and measure no angle in the complex plane.

The complex plane also allows a complex step size, which rotates the update and extends the search from a ray to a half-plane: [32] show that the complex step size approximates Hessian information better than a real one, and [33] and [34] adapt it during training. The authors of [35] propose to adjust a complex step size, shared by all parameters, according to its phase, obtained by comparing gradient norms at three trial points; the angle thus comes from flatness rather than from consecutive updates, at the cost of three extra gradient evaluations per step. Methods based on curvature or line searches are also more expensive per step: HCSCGM [36] and its extension with an adaptive complex step size, HCSCGACS [37], rely on complex inexact line searches, AS-CNAG [38] sets the step from an approximate Hessian, and [39], [40]

Corresponding author: Enrico Ballini (e-mail: enrico.ballini@gmail.com).
[1]Aarhus University, Aarhus, Denmark.
[2]Technical University of Denmark, Kongens Lyngby, Denmark.

develop adaptive complex L-BFGS methods. We therefore focus on optimizers that use one stochastic gradient per step and inexpensive vector operations.

This work builds on the idea that the angle between consecutive updates is an informative signal for step-size control in CVNNs. We propose AURA, a per-parameter step-size multiplier applicable on top of any optimizer, which enlarges the step when consecutive update directions are aligned and of similar length and reduces it when they are opposed or keep rotating in the same sense. We combine AURA with Adam [28] and Muon [41] and evaluate it on four problems with fully connected neural networks, from scalar complex functions to more complicated physics-informed training. Unlike real-valued rules, which can only test whether consecutive updates agree in sign and magnitude, AURA also measures how much they rotate in the complex plane, in which sense they rotate, and how similar they are in magnitude. The building blocks of AURA have already been used in the cited literature, but their combination for complex parameters is new. Section II presents the AURA algorithm, Section III relates it to the closest optimizers, Section IV reports the case studies, and Section V concludes.

II. AURA

The method we propose scales the update direction of a base optimizer by a per-parameter multiplier, which, in a nutshell, increases when consecutive directions agree in the complex plane and decreases when they do not. This section fixes the notation, states the algorithm, and explains the building blocks of the algorithm.

a) Notation: Let $w_{j,t} = x_{j,t} + \mathrm{i}\, y_{j,t} \in \mathbb{C}$, $j = 1, \dots, N$, be the N trainable parameters at iteration $t \in \mathbb{N}$, and $\mathscr{L}_t : \mathbb{C}^N \to \mathbb{R}$ the loss at step t, e.g., the mini-batch loss (12). Being real-valued, $\mathscr{L}_t$ is not holomorphic and is differentiated as a map $\mathbb{R}^{2N} \to \mathbb{R}$ under $\mathbb{C}^N \simeq \mathbb{R}^{2N}$. The gradient

$$g_{j,t} = \frac{\partial \mathscr{L}_t}{\partial x_{j,t}} + \mathrm{i}\, \frac{\partial \mathscr{L}_t}{\partial y_{j,t}}, \tag{1}$$

combines the two real partial derivatives into a single complex number, and $-g_{j,t}$ is the steepest-descent direction of $\mathscr{L}_t$ in $w_{j,t}$.

b) Algorithm: Given the direction $d_{j,t}$ of a base optimizer $\mathcal{G}$, AURA computes a multiplier $\gamma_{j,t} > 0$ and takes the step $\alpha\gamma_{j,t}d_{j,t}$. With the initial values $d_{j,0} = 0$, $\tilde{\zeta}_{j,0} = 0$, and $\gamma_{j,0} = 1$, and suitable choices for the hyperparameters listed in Table I and discussed below, the AURA algorithm takes the following form for every j and $t \geq 1$:

$$\text{Optimizer (Adam, Muon, \dots)} \qquad d_{j,t} = \mathcal{G}_j\big(\{g_{k,s}\}_{k\leq N,\, s\leq t}\big), \tag{2a}$$

$$\text{Consistency variable} \begin{cases} \zeta_{j,t} = \dfrac{2\, d_{j,t}\overline{d_{j,t-1}}}{|d_{j,t}|^2 + |d_{j,t-1}|^2 + \varepsilon_{\mathrm{E}}}, & |\zeta_{j,t}| < 1, \\ \tilde{\zeta}_{j,t} = \beta_\zeta \tilde{\zeta}_{j,t-1} + (1-\beta_\zeta)\zeta_{j,t}, & \widehat{\zeta}_{j,t} = \dfrac{\tilde{\zeta}_{j,t}}{1-\beta_\zeta^{\,t}}, \\ \chi_{j,t} = \mathrm{Re}\big(\widehat{\zeta}_{j,t}\big) \in (-1,1), \\ \Psi_{j,t} = \mathrm{Im}\big(\widehat{\zeta}_{j,t}\big) \in (-1,1), \end{cases} \tag{2b}$$

$$\text{“Delta-bar-delta”-style } \alpha \text{ adjustment} \begin{cases} \sigma_{j,t} = \big(\chi_{j,t} \leq \chi_{\mathrm{o}}\big) \text{ or } \big(|\Psi_{j,t}| \geq \Psi_{\mathrm{o}}\big), \\ \rho_{j,t} = \big(\chi_{j,t} \geq \chi_{\mathrm{a}}\big) \text{ and } \big(|\Psi_{j,t}| \leq \Psi_{\mathrm{a}}\big), \\ \gamma_{j,t} = \begin{cases} \max\{\eta_- \gamma_{j,t-1}, \gamma_{\min}\}, & \sigma_{j,t} = 1, \\ \min\{\eta_+ \gamma_{j,t-1}, \gamma_{\max}\}, & \rho_{j,t} = 1, \\ \gamma_{j,t-1}, & \text{otherwise}, \end{cases} \end{cases} \tag{2c}$$

$$w_{j,t+1} = w_{j,t} - \alpha\gamma_{j,t} d_{j,t} - \alpha\lambda w_{j,t}. \tag{2d}$$

c) Base optimizer: Line (2a) is any first-order optimizer, e.g., Adam [28] or Muon [41]; the subsequent blocks depend on $d_{j,t}$ only. With $\gamma_{j,t} = 1$ for all j and t, AURA reduces to $\mathcal{G}$, which shows that the proposed algorithm acts as a plug-in to the base optimizer.

d) Consistency variable: Block (2b) compares $d_{j,t}$ with $d_{j,t-1}$ through the complex variable $\zeta_{j,t}$, a Dice-like measure [42] (Section A). Under $\mathbb{C} \simeq \mathbb{R}^2$, the real part of its numerator is the inner product of the two directions and the imaginary part their signed area; the denominator normalizes by their squared lengths, so that $\zeta_{j,t} \to 1$ for equal directions, $\zeta_{j,t} \to -1$ for opposite ones, $\zeta_{j,t} \to e^{\mathrm{i}\theta}$ for a rotation by θ at equal length, and $|\zeta_{j,t}|$ decreases as the two lengths separate. Unlike a cosine similarity, $\zeta_{j,t}$ therefore captures length, angle, and sense of rotation simultaneously; the safeguard $\varepsilon_{\mathrm{E}} > 0$ only prevents division by zero. The moving average $\tilde{\zeta}_{j,t}$, with decay rate β_ζ, retains the memory of past directions and, since it starts from zero, is bias-corrected. The real part $\chi_{j,t}$ of $\widehat{\zeta}_{j,t}$ is the smoothed alignment of consecutive directions, the imaginary part $\Psi_{j,t}$ their smoothed signed rotation: rotations of the same sense accumulate in $\Psi_{j,t}$, alternating ones cancel.

e) Multiplier and update: Block (2c) converts the two statistics into $\gamma_{j,t}$ by a three-way test, conceptually similar to the strategies delta-bar-delta [43] and RPROP [44]: the multiplier is increased by $\eta_+ > 1$ when the flag $\rho_{j,t}$ holds, decreased by $\eta_- \in (0,1)$ when $\sigma_{j,t}$ holds, and left unchanged otherwise, always within $[\gamma_{\min}, \gamma_{\max}]$. Alignment alone is not sufficient for an increase: a sequence of small rotations of constant sense keeps $\chi_{j,t}$ close to 1 but accumulates in $\Psi_{j,t}$. An increase therefore requires both $\chi_{j,t} \geq \chi_{\mathrm{a}}$ and $|\Psi_{j,t}| \leq \Psi_{\mathrm{a}}$, whereas either $\chi_{j,t} \leq \chi_{\mathrm{o}}$ or $|\Psi_{j,t}| \geq \Psi_{\mathrm{o}}$ forces a decrease;

TABLE I
HYPERPARAMETERS OF AURA

Symbol	Range	Role
Consistency variable (2b)		
β_ζ	$[0, 1)$	Decay rate of the average $\tilde{\zeta}_{j,t}$
ε_{E}	> 0	Safeguard on the denominator of $\zeta_{j,t}$
"Delta-bar-delta"-style α adjustment (2c)		
χ_{a}	$(\chi_{\mathrm{o}}, 1]$	Alignment threshold for growth
χ_{o}	$[-1, \chi_{\mathrm{a}})$	Opposition threshold forcing decrease
Ψ_{a}	$[0, \Psi_{\mathrm{o}})$	Rotation tolerance for growth
Ψ_{o}	$(\Psi_{\mathrm{a}}, 1]$	Rotation threshold forcing decrease
η_-	$(0, 1)$	Decrease factor of $\gamma_{j,t}$
η_+	> 1	Increase factor of $\gamma_{j,t}$
$\gamma_{\min}$	$(0, 1]$	Floor on the multiplier
$\gamma_{\max}$	≥ 1	Ceiling on the multiplier
Update (2d)		
λ	≥ 0	Decoupled (AdamW) weight decay

the two flags are exclusive since $\chi_{\mathrm{o}} < \chi_{\mathrm{a}}$ and $\Psi_{\mathrm{a}} < \Psi_{\mathrm{o}}$. The update (2d) applies the scaled direction with base step size $\alpha > 0$ and decoupled weight decay $\lambda \geq 0$, as in AdamW [45].

f) Hyperparameters and cost: AURA adds the ten hyperparameters of Table I, plus the weight decay λ; the numerical values used in the test cases (Section IV) are given in Table II. Most have a clear interpretation and should not require heavy tuning: ε_{E} is a safeguard, $\gamma_{\min}$ and $\gamma_{\max}$ bound the multiplier, and $\eta_\pm$ set its rate of change. The decay rate β_ζ and the four thresholds can be more difficult to tune. Per parameter, AURA stores $d_{j,t-1}$, $\tilde{\zeta}_{j,t}$, and $\gamma_{j,t}$, i.e., five real numbers against the three of Adam (a complex first moment and a real second moment), and adds a few elementwise operations per step; it needs no extra gradient evaluations. The measured overhead is reported in Section IV.

III. RELATED WORK

This section relates AURA to the optimizers it builds on or resembles, with a focus on how each method measures the agreement between successive gradients or updates and on how it uses that measurement.

a) Adam and Muon: Adam [28] and Muon [41] are the base optimizers $\mathcal{G}$ used in this work because they are established methods with known good performance, although other optimizers could be chosen. Adam normalizes the first moment of the gradient by the square root of its second moment, which Balles and Hennig [46] interpret as a damping of each step by the signal-to-noise ratio (SNR) of the corresponding gradient component. Muon applies momentum and orthogonalizes the update of each weight matrix by Newton–Schulz iterations; it carries no per-parameter variance estimate. In both cases, the SNR does not enter $\zeta_{j,t}$, which depends only on the directions $d_{j,t}$ and $d_{j,t-1}$: AURA inherits whatever variance adaptation the base optimizer provides and adds none of its own. This could represent a limitation for noisy updates, since $\zeta_{j,t}$ then measures the agreement between noisy directions (see also Section IV-D).

b) Step-size adaptation by sign information: The idea of adapting the step size from the agreement of successive derivatives dates back to the rules of Kesten, Saridis, and Barto and Sutton [47]–[49], and was systematized by Jacobs [43] in the delta-bar-delta rule, which increases the step size of a weight while consecutive derivatives share their sign and decreases it when they differ. AURA rests on the same principle, with the sign test replaced by a test on the complex quantity $\widehat{\zeta}_{j,t}$.

c) RPROP: The method proposed by [44], [50] assigns each weight its own update value, which is increased by a factor $\eta_+ > 1$ while the partial derivative keeps its sign and decreased by a factor $\eta_- < 1$ when the sign changes; the weight then moves by this value against the sign of the derivative, irrespective of its magnitude. As in AURA, the agreement of successive derivatives sets the size of the next step. Complex RPROP [51], [52] applies the same rule to the real and imaginary parts separately, and therefore measures no angle in the complex plane.

d) CoRe: The CoRe optimizer [11], [12] combines an Adam direction $u_{j,t} \in \mathbb{R}$, the counterpart of $d_{j,t}$ in (2a), with an RPROP-style per-weight step size $s_{j,t} \in \mathbb{R}$, as AURA does, and differs from it in three respects. First, it is defined for real-valued networks: its step-size rule depends only on the sign of the product of two consecutive first moments, the real-domain counterpart of the sign of $\chi_{j,t}$. Second, the rule is gated by a binary plasticity factor $P_{j,t} \in \{0, 1\}$, which freezes the weights with a high importance score. The update reads

$$w_{j,t} = w_{j,t-1} - \underbrace{u_{j,t}\, P_{j,t}\, s_{j,t}}_{\text{update}} - \underbrace{d_\chi\, |u_{j,t}|\, P_{j,t}\, s_{j,t} w_{j,t-1}}_{\text{regularization}}, \quad (3)$$

where $d_\chi \in \mathbb{R}_{\geq 0}$ is a hyperparameter. Since $P_{j,t}$ also multiplies the update, a weight can remain unchanged for a whole step even when $u_{j,t}$ is nonzero. AURA has no such switch: $\gamma_{j,t}$ is bounded below by $\gamma_{\min} > 0$, so the step in (2d) is never suppressed while $d_{j,t}$ is nonzero. Third, the regularization term in (3) is a weight decay proportional to the magnitude $|u_{j,t}| P_{j,t} s_{j,t}$ of the update, with a group-specific coefficient d_χ, so that stable weights are decayed the least. Overfitting control is not a target of AURA, which uses the decoupled

weight decay of Loshchilov and Hutter [45] and is compatible with other strategies.

e) AdaRem: AdaRem [53] multiplies the step size of each parameter by $a_{t,i} = 1 + \lambda_t b_{t,i}$, where $b_{t,i} = g_{t,i} m_{t,i} / (|g_{t,i}| \max |m_t| + \epsilon)$ and m_t is the exponential moving average of the gradient. Up to ϵ, $b_{t,i} = \operatorname{sign}(g_{t,i} m_{t,i}) \, |m_{t,i}| / \max |m_t|$: for two scalars, Salton's cosine reduces to the sign alone, which AdaRem weights by the relative size of the momentum. As in AURA, the step increases when the current gradient agrees with the past direction and decreases when it opposes it.

f) TAM: The algorithm TAM [54] computes the cosine similarity (Salton's measure, Section A) between the previous momentum direction and the current gradient, an ingredient close to $\chi_{j,t}$; this quantity, however, modifies the momentum rather than the step size.

g) diffGrad: diffGrad [4] multiplies the Adam direction by a "friction coefficient" $\xi_{j,t} = \left(1 + e^{-|g_{j,t} - g_{j,t-1}|}\right)^{-1} \in [\frac{1}{2}, 1)$, so that "large changes in the gradient incur less friction, whereas small changes in the gradient incur more friction" [4]. Its behavior is therefore, to some extent, opposite to that of AURA.

h) AngularGrad: AngularGrad [3] rescales the Adam direction by an angle-like measure of consecutive gradients, the same kind of quantity that drives AURA. The multiplier is $\phi_{j,t}^{\mathrm{Ang}} = \frac{1}{2} + \frac{1}{2} \tanh \left| f(A_{j,\min}) \right|$, where $A_{j,t} = \arctan \left| (g_{j,t} - g_{j,t-1}) / (1 + g_{j,t} g_{j,t-1}) \right|$ is the angle between two consecutive scalar gradients interpreted as slopes, $A_{j,\min} = \min\{A_{j,t-1}, A_{j,t}\}$, and f is either the cosine or the tangent. Three differences from AURA stand out. First, AngularGrad is designed and validated on real-valued networks only. Second, $A_{j,t}$ is an unoriented slope angle, rather than the oriented phase difference of complex updates. Third, $A_{j,t}$ hardly separates concordant from opposite gradients, because it compares slopes and a sign change of a small gradient is a small angle: with $f = \cos$, the pair $g_{j,t-1} = -0.01$, $g_{j,t} = 0.01$ gives $A_{j,t} = 0.02$, two equal gradients give $A_{j,t} = 0$, and both lead to $\phi_{j,t}^{\mathrm{Ang}} \approx 0.881$.

i) Hypergradient descent: Hypergradient descent [6] differentiates the loss with respect to the step size. Although conceptually distant from AURA, its multiplicative variant shares an important ingredient. As reported by Baydin et al. [6], the multiplicative rule is

$$\alpha_t = \alpha_{t-1} \left(1 - \beta' \cos \phi_t\right), \tag{4}$$

where

$$\cos \phi_t = \frac{\widetilde{\nabla} \mathscr{L}(w_{t-1})^\top \nabla_\alpha u(\mathcal{W}_{t-2}, \alpha_{t-1})}{\left\| \widetilde{\nabla} \mathscr{L}(w_{t-1}) \right\| \left\| \nabla_\alpha u(\mathcal{W}_{t-2}, \alpha_{t-1}) \right\|}. \tag{5}$$

Here $\beta' \in \mathbb{R}$ is a hyperparameter, $\mathscr{L} \colon \mathbb{R}^N \to \mathbb{R}$ is the loss, $\widetilde{\nabla} \mathscr{L}$ its stochastic estimator, and u the update rule of the base method, $w_t = u(\mathcal{W}_{t-1}, \alpha) \in \mathbb{R}^N$ with $\mathcal{W}_t = \{w_i\}_{i=0}^t$. The step size is thus updated according to the angle between two vectors, as in AURA; the vectors, however, differ in meaning. Since $\nabla_\alpha u(\mathcal{W}_{t-2}, \alpha_{t-1})$ is the previous update direction, hypergradient descent correlates the current gradient with the previous update, whereas AURA correlates two consecutive update directions.

j) Phase of the complex step size: FGACGD [35] also relies on an angle, but of a different kind: the phase $\phi_t \in [-\pi/4, \pi/4]$ of a single complex step size $|\eta_t| e^{\mathrm{i}\phi_t}$ shared by all trainable parameters. The phase is selected by comparing the gradient norms at three trial points, reached with step sizes $\rho e^{\mathrm{i}0}$ and $\rho e^{\pm \mathrm{i}\pi/4}$, and steers the update toward flatter regions; no angle between consecutive directions is measured.

k) Summary: The methods above measure the agreement of successive gradients or updates in two ways. Sign-based rules [11], [43], [44] detect only whether the direction is reversed, and angle-based rules [3], [6], [54] detect an unoriented angle computed from real-valued gradients. AURA operates per parameter in the complex plane and measures how much consecutive directions differ – in length and in angle – and in which sense they rotate. This combination is the novelty claimed here; its ingredients are established: per-parameter multiplicative step control [44], angle-sensitive modulation [3], [6], and the combination of Adam with RPROP [11].

IV. Case studies

This section compares AURA with the baseline optimizers on four test cases, selected to span different common learning tasks, architectures, and training regimes rather than to reach the best possible performance on any of them. The first three approximate a non-holomorphic (Section IV-C), a holomorphic (Section IV-D), and a multivariate (Section IV-E) complex function with fully connected networks; they are inexpensive and serve for sensitivity studies on the step size, the moment decay rates, the mini-batch size, and the floating-point precision. The last one tests whether the conclusions carry over to a physics-informed loss with full-batch training (Section IV-F). Since the object of study is the optimizer and not the task, reference architectures and standard setups are used throughout.

The protocol follows, as far as practicable, the recommendations of [13], [14], [55] on the fair comparison of optimizers. Within each case, all methods start from the same initialization, are trained on the same data, and are run for several random seeds; they are compared through the training and test loss curves and the three scalar metrics of Section IV-A: the best training loss reached, the area under the logarithmic learning curve, and the wall-clock time per step relative to plain stochastic gradient descent. Since the hyperparameter search space can by itself determine the ranking of optimizers [15], the hyperparameters of AURA are fixed, although their number would allow per-case tuning; this choice penalizes AURA, if anything, and tests at the same time whether the method requires such tuning. Section IV-F complements this with a comparison in which every optimizer is tuned under limited budget.

Four cases cannot cover every training scenario. The aim is to show that AURA is advantageous on a reasonably broad set of problems under a fair protocol, to quantify its overhead, and to document a failure mode (Section IV-D); extending the comparison to further tasks and larger models is left to future work.

All the test cases were run on a single A100 40GB graphics processing unit (GPU).

A. Metrics for comparison

In each test case, every method m of the compared set $\mathcal{M}$ is trained from the same initialization and on the same data, independently for N_{seed} random seeds $s = 1, \ldots, N_{\text{seed}}$; the value of N_{seed} is given in the settings table of each case. Each run records the training loss $\mathscr{L}_{\text{train}}^{(m,s)}(t)$ at every optimizer step t, namely the loss of the current mini-batch, or of the whole training set in the full-batch case, and the test-set loss $\mathscr{L}_{\text{test}}^{(m,s)}(t)$ at regular intervals. Methods are compared through the loss curves and through three scalar metrics, both summarized over seeds by the median and the 25th and 75th percentiles Q_{25} and Q_{75}, computed by linear interpolation; the tables report a metric with median x as $x^{+(Q_{75}-x)}_{-(x-Q_{25})}$. Steps at which the loss is not finite, which occur only in diverged runs, are excluded from the scalar metrics but highlighted in the results summary tables. The three metrics are listed below.

- **Min training loss.** The best training loss reached within a run, $\mathscr{L}_{\text{min}}^{(m,s)} = \min_t \mathscr{L}_{\text{train}}^{(m,s)}(t)$, summarized over seeds by its median and percentiles,

$$\mathscr{L}_{\text{min}}^{(m)} = \underset{s=1,\ldots,N_{\text{seed}}}{\text{median}} \mathscr{L}_{\text{min}}^{(m,s)}. \tag{6}$$

  It summarizes a run by a single number that does not depend on the choice of a stopping step. It is used to assess how effectively the optimizer minimizes the loss; the median reduces the influence of unusually favorable or unfavorable runs.
- **Area under the log learning curve.** To account for the whole trajectory rather than for its best point only, the training-loss curve is averaged on a base-10 logarithmic scale,

$$A^{(m,s)} = \frac{1}{T+1} \sum_{t=0}^{T} \Big(C + \log_{10} \mathscr{L}_{\text{train}}^{(m,s)}(t) \Big), \qquad C = 10, \tag{7}$$

  where T is the final optimizer step, common to all methods within a case. $A^{(m,s)}$ is the normalized area between the log learning curve and a floor at loss 10^{-C}, placed below the smallest loss reached in any case so that the area is non-negative; equivalently, it is C plus the base-10 logarithm of the geometric-mean training loss, and the constant C affects no comparison. Since every decade of loss reduction has the same weight, $A^{(m,s)}$ rewards methods that reduce the loss early and keep it low, and is not dominated by the large losses of the initial transient. This metric is used as it provides a quantitative assessment of the whole training curve. A complementary qualitative assessment is provided by the training-curve plots.
- **Training time.** The wall-clock time $\tau^{(m,s)}$ of the training loop, excluding the diagnostic instrumentation, such as the routines that store the data from which the figures are generated, and the just-in-time compilation of the first step. Since a time in seconds is tied to the GPU it was measured on, following [13], we use the ratio

$$\tilde{\tau}^{(m,s)} = \tau^{(m,s)} / \tau^{(\text{SGD})}, \tag{8}$$

  summarized over seeds by its median $\tilde{\tau}^{(m)}$ and percentiles, where $\tau^{(\text{SGD})}$ is the median time of the plain stochastic gradient descent update, $w_{j,t+1} = w_{j,t} - \alpha\, g_{j,t}$, over $N_{\text{SGD}} = 5$ seeds, run with the same architecture, precision, step budget, and GPU. As SGD is the cheapest first-order step, $\tilde{\tau}^{(m)}$ is the (almost) hardware-independent factor by which method m is slower than it. A separate SGD baseline is measured for each setting whose per-step cost differs, i.e., for each architecture and for `complex64` versus `complex128`. SGD serves only as a unit of time and is not part of $\mathcal{M}$. We adopt this metric because the actual training time is an important practical feature of the optimizer reflecting its computational efficiency.

B. Optimizer settings

The baseline optimizers are chosen as follows. RPROP [44] is included because, as in AURA, the agreement of successive derivatives sets the size of the next step. Adam [28] and NadamW, that is, Nadam [31] with decoupled weight decay [45], are included because they are widely used and perform well for training complex-valued networks [17]; Adam is, in addition, the base optimizer of Adam-AURA, so that the comparison between the two isolates the effect of AURA. CvAMSGrad, the split-complex extension of AMSGrad [29] proposed in [17], is included because it attains a lower steady-state error than SGD on the two complex-valued benchmark problems of [17], at a moderate increase in computational cost. Muon [41] is included because it is designed specifically for the weight matrices of the hidden layers of neural networks, and it is the base optimizer of Muon-AURA.

The default hyperparameters for each optimizer are listed in Table II. Where applicable, these are adjusted as described in the relevant test case. The ten hyperparameters specific to AURA are held fixed across all test cases, except in Section IV-F, where its principal ones are tuned together with those of the baselines. The resulting comparison is pragmatic and fair: if anything, this constraint disadvantages AURA, which nonetheless outperforms the baselines in many cases.

C. TEST 1: univariate scalar complex non-holomorphic function

a) Rationale: This test provides a lightweight analytical benchmark for sensitivity and ablation studies. The goal is to approximate a non-holomorphic function, complementing the holomorphic case considered in a separate test. We empirically investigate the sensitivity of the optimizers to their main hyperparameters by comparing their performance across two fully connected neural network architectures, different step sizes, and different exponential moving average (EMA) decay rates.

b) Learning task: The domain is the square $\Omega_\zeta = \{\zeta \in \mathbb{C} : |\text{Re}\zeta| \le 1,\ |\text{Im}\zeta| \le 1\}$. The target function is

$$\begin{aligned} f(\zeta) = {} & \exp\big((0.30 - 0.20\text{i})\,\zeta\overline{\zeta}\big) + 0.25 \sin(\zeta)\cos(\overline{\zeta}) \\ & + 0.10\,\zeta^2\overline{\zeta} + 0.08\,\zeta\overline{\zeta}^2 - 0.05\text{i}\,(\zeta\overline{\zeta})^2. \end{aligned} \tag{9}$$

TABLE II
DEFAULT HYPERPARAMETERS USED IN THE BENCHMARK CASES IN SECTION IV.

Optimizer	Hyperparameters
RPROP	$\eta_- = 0.5$, $\eta_+ = 1.2$, $\Delta \in [1 \times 10^{-6}, 50]$; $\Delta_0 = \alpha$
Adam	$\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon_{\mathrm{A}} = 1 \times 10^{-8}$
Adam (variable LR)	Same $\beta_1, \beta_2, \varepsilon_{\mathrm{A}}$ as Adam; learning rate α for the first 50% of updates, then $\alpha/10$
NadamW	$\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon_{\mathrm{A}} = 1 \times 10^{-8}$, weight decay: 1×10^{-4}
CvAMSGrad	$\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon_{\mathrm{A}} = 1 \times 10^{-8}$; bias correction included
Muon	$\beta = 0.95$, Newton–Schulz steps: 5, Nesterov: yes, weight decay: 0; matrix step $10\,\alpha$; bias fallback AdamW at $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon_{\mathrm{A}} = 1 \times 10^{-8}$
Adam-AURA	$\beta_\zeta = 0.95$, $\varepsilon_{\mathrm{E}} = 1 \times 10^{-6}$, $\chi_{\mathrm{a}} = 0.7$, $\chi_{\mathrm{o}} = 0.4$, $\Psi_{\mathrm{a}} = 0.015$, $\Psi_{\mathrm{o}} = 0.3$, $\eta_- = 0.99$, $\eta_+ = 1.01$, $\gamma \in [1 \times 10^{-3}, 1000]$, $\lambda = 1 \times 10^{-4}$
Muon-AURA	$\beta = 0.95$, Newton–Schulz steps: 5, Nesterov: yes, matrix step $10\,\alpha$; $\beta_\zeta = 0.95$, $\varepsilon_{\mathrm{E}} = 1 \times 10^{-6}$, $\chi_{\mathrm{a}} = 0.75$, $\chi_{\mathrm{o}} = 0.4$, $\Psi_{\mathrm{a}} = 0.01$, $\Psi_{\mathrm{o}} = 0.2$, $\eta_- = 0.99$, $\eta_+ = 1.01$, $\gamma \in [1 \times 10^{-3}, 1000]$, $\lambda = 1 \times 10^{-4}$; bias direction Adam at $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon_{\mathrm{A}} = 1 \times 10^{-8}$

The training and test sets consist of mutually independent samples drawn uniformly from Ω_ζ:

$$\mathcal{Z}_{\mathrm{train}} = \{\zeta_i\}_{i=1}^{n_{\mathrm{train}}}, \quad \zeta_i \overset{\text{i.i.d.}}{\sim} \mathrm{Unif}(\Omega_\zeta), \tag{10}$$

$$\mathcal{Z}_{\mathrm{test}} = \{\zeta'_i\}_{i=1}^{n_{\mathrm{test}}}, \quad \zeta'_i \overset{\text{i.i.d.}}{\sim} \mathrm{Unif}(\Omega_\zeta), \qquad \mathcal{Z}_{\mathrm{test}} \cap \mathcal{Z}_{\mathrm{train}} = \emptyset, \tag{11}$$

with $n_{\mathrm{test}} = \mathrm{round}(0.2\, n_{\mathrm{train}})$. The real and imaginary parts of each sample are drawn independently from $\mathrm{Unif}([-1, 1])$. Denoting the neural network by $f_{\mathcal{W}}$ and a nonempty mini-batch by $\mathcal{B} \subseteq \mathcal{Z}_{\mathrm{train}}$, training minimizes the mean squared error

$$\mathscr{L}_{\mathcal{B}}(\mathcal{W}) = \frac{1}{|\mathcal{B}|} \sum_{\zeta \in \mathcal{B}} \left| f_{\mathcal{W}}(\zeta) - f(\zeta) \right|^2. \tag{12}$$

c) Network architectures: The networks are compositions of affine maps and componentwise nonlinear activations:

$$f_{\mathcal{W}} = A_L \circ \varsigma \circ A_{L-1} \circ \cdots \circ \varsigma \circ A_1,$$

with $A_\ell(z) = W_\ell z + b_\ell$, $\ell = 1, \ldots, L$, where $W_\ell \in \mathbb{C}^{n_\ell \times n_{\ell-1}}$, $b_\ell \in \mathbb{C}^{n_\ell}$, $n_0 = n_L = 1$, and $\mathcal{W} = \{W_\ell, b_\ell\}_{\ell=1}^{L}$. The activation

$$\varsigma(\zeta) = \mathrm{SiLU}(\mathrm{Re}\zeta) + \mathrm{i}\ \mathrm{SiLU}(\mathrm{Im}\zeta),$$

is applied componentwise in the hidden layers. This activation is non-holomorphic, so the networks are generally non-holomorphic.

The complete benchmark configuration is given in Table III.

TABLE III
CONFIGURATION OF THE NON-HOLOMORPHIC OPTIMIZER BENCHMARK.

Setting	Value
Primary architecture	$(1, 32, 32, 32, 32, 1)$, 3,265 trainable parameters
Secondary architecture	$(1, 128, 128, 128, 128, 128, 1)$, 66,433 trainable parameters
Hidden activation	$\mathrm{SiLU}(\mathrm{Re}\zeta) + \mathrm{i}\,\mathrm{SiLU}(\mathrm{Im}\zeta)$
Training / test set	2,500 / 500 points (test: 20%)
Mini-batch size	256 points
Updates / seeds	12,000 / $\{0, \ldots, 4\}$
Arithmetic	`complex64` (32-bit real components)
Initialization	Glorot/Xavier normal rule [56], applied separately to the real and imaginary parts; zero biases
Base learning rate α (primary)	5×10^{-4}
Secondary-architecture learning rate	5×10^{-5}
Optimizer hyperparameters	default values; see Table II

d) Results – α sensitivity: Table IV reports the three metrics for the step sizes $\alpha/10$, α, and 10α, and Figure 1 shows the corresponding training- and test-loss curves for both architectures. Over this 100-fold range, $\mathscr{L}_{\min}^{(m)}$ varies by more than two orders of magnitude for Adam and its variants and by more than three for Muon at the primary architecture and by more than one at the secondary, but by less than one for Adam-AURA, and it remains of order 10^{-7} or below for Muon-AURA. RPROP is similarly insensitive to α, but its minimum loss is 6 to 50 times that of Adam-AURA. Muon-AURA attains the lowest $\mathscr{L}_{\min}^{(m)}$ and $A^{(m)}$ in all six settings; since $A^{(m)}$ is C plus the base-10 logarithm of the geometric-mean training loss (7), its margin of about 2 to 3 over the best baseline corresponds to a loss roughly two to three orders of magnitude lower along the whole trajectory, not only at its best point. This improvement comes at a small cost: the time ratio (8) of Adam-AURA exceeds that of Adam by at most 0.1, within the spread over seeds, and that of Muon-AURA is even slightly below that of Muon, so the higher ratio of Muon-AURA, up to

TABLE IV
STEP-SIZE SENSITIVITY FOR THE NON-HOLOMORPHIC BENCHMARK.

Arch.	Optimizer	$\mathscr{L}_{\min}^{(m)}$ ($\times 10^{-6}$)			$A^{(m)}$			Training time ($\times$SGD)
		$\alpha/10$	α	10α	$\alpha/10$	α	10α	α
Primary	RPROP	34^{+19}_{-8}	203^{+6}_{-93}	41^{+6}_{-14}	$5.9^{+0.3}_{-0.1}$	$6.6^{+0.01}_{-0.2}$	$6.1^{+0.1}_{-0.3}$	$1.67^{+0.0002}_{-0.02}$
	Adam	167^{+92}_{-52}	$4.6^{+0.5}_{-0.1}$	$1.3^{+0.1}_{-0.2}$	$7.3^{+0.1}_{-0.1}$	$5.8^{+0.1}_{-0.03}$	$5.4^{+0.01}_{-0.04}$	$1.59^{+0.005}_{-0.04}$
	Adam (variable LR)	614^{+106}_{-116}	$7.6^{+2.5}_{-0.4}$	$1.5^{+0.2}_{-0.03}$	$7.4^{+0.1}_{-0.1}$	$5.8^{+0.1}_{-0.01}$	$5.1^{+0.002}_{-0.001}$	$\mathbf{1.580^{+0.006}_{-0.012}}$
	NadamW	151^{+99}_{-49}	$5.6^{+0.7}_{-0.2}$	$1.1^{+0.1}_{-0.02}$	$7.3^{+0.1}_{-0.1}$	$5.9^{+0.1}_{-0.05}$	$5.8^{+0.02}_{-0.03}$	$1.59^{+0.04}_{-0.01}$
	CvAMSGrad	893^{+119}_{-94}	$16^{+0.3}_{-4}$	$2.2^{+0.3}_{-0.02}$	$7.4^{+0.05}_{-0.002}$	$6.1^{+0.05}_{-0.1}$	$5.1^{+0.01}_{-0.04}$	$1.6^{+0.1}_{-0.2}$
	Muon	$0.2^{+0.4}_{-0.01}$	$32^{+3}_{-0.4}$	283^{+60}_{-44}	$5.3^{+0.02}_{-0.003}$	$6.0^{+0.01}_{-0.002}$	$7.0^{+0.001}_{-0.01}$	$1.88^{+0.0002}_{-0.02}$
	Adam-AURA	$3.0^{+2.1}_{-0.4}$	$3.9^{+0.1}_{-1.9}$	$1.7^{+0.8}_{-0.04}$	$5.3^{+0.1}_{-0.1}$	$5.1^{+0.1}_{-0.1}$	$4.8^{+0.03}_{-0.1}$	$1.64^{+0.11}_{-0.01}$
	Muon-AURA	$\mathbf{0.1^{+0.003}_{-0.002}}$	$\mathbf{0.04^{+0.004}_{-0.005}}$	$\mathbf{0.01^{+0.001}_{-0.0001}}$	$\mathbf{3.4^{+0.1}_{-0.003}}$	$\mathbf{3.1^{+0.04}_{-0.1}}$	$\mathbf{2.7^{+0.003}_{-0.01}}$	$1.80^{+0.09}_{-0.03}$
Secondary	RPROP	$9.2^{+19.1}_{-0.2}$	16^{+4}_{-2}	17^{+1}_{-1}	$5.4^{+0.4}_{-0.04}$	$5.7^{+0.03}_{-0.1}$	$5.6^{+0.1}_{-0.05}$	$1.70^{+0.02}_{-0.01}$
	Adam	1505^{+4}_{-103}	28^{+4}_{-8}	$3.4^{+0.2}_{-0.1}$	$8.0^{+0.01}_{-0.04}$	$6.8^{+0.02}_{-0.05}$	$5.9^{+0.01}_{-0.1}$	$1.65^{+0.08}_{-0.01}$
	Adam (variable LR)	1786^{+21}_{-6}	305^{+12}_{-23}	$4.4^{+0.3}_{-0.6}$	$8.0^{+0.01}_{-0.04}$	$7.0^{+0.02}_{-0.02}$	$5.7^{+0.01}_{-0.2}$	$\mathbf{1.64^{+0.01}_{-0.02}}$
	NadamW	1494^{+9}_{-93}	27^{+1}_{-5}	$3.7^{+0.8}_{-0.01}$	$8.0^{+0.01}_{-0.04}$	$6.8^{+0.02}_{-0.03}$	$6.1^{+0.01}_{-0.03}$	$1.64^{+0.01}_{-0.02}$
	CvAMSGrad	1785^{+16}_{-24}	287^{+18}_{-14}	$4.1^{+0.2}_{-0.7}$	$7.9^{+0.01}_{-0.05}$	$7.0^{+0.04}_{-0.01}$	$5.7^{+0.03}_{-0.1}$	$1.66^{+0.002}_{-0.02}$
	Muon	50^{+6}_{-14}	$3.5^{+0.3}_{-0.3}$	$55^{+0.1}_{-0.3}$	$8.4^{+0.01}_{-0.1}$	$5.2^{+0.02}_{-0.01}$	$6.1^{+0.01}_{-0.01}$	$2.218^{+0.010}_{-0.0003}$
	Adam-AURA	$1.5^{+0.1}_{-0.2}$	$1.6^{+0.2}_{-0.4}$	$0.6^{+0.2}_{-0.2}$	$5.4^{+0.01}_{-0.1}$	$4.9^{+0.2}_{-0.01}$	$4.6^{+0.1}_{-0.01}$	$1.68^{+0.05}_{-0.05}$
	Muon-AURA	$\mathbf{0.02^{+0.001}_{-0.001}}$	$\mathbf{0.01^{+0.0004}_{-0.0003}}$	$\mathbf{0.01^{+0.0004}_{-0.000004}}$	$\mathbf{2.9^{+0.03}_{-0.02}}$	$\mathbf{2.8^{+0.02}_{-0.01}}$	$\mathbf{2.6^{+0.01}_{-0.04}}$	$2.147^{+0.015}_{-0.003}$

2.1, stems from the Muon direction rather than from AURA.

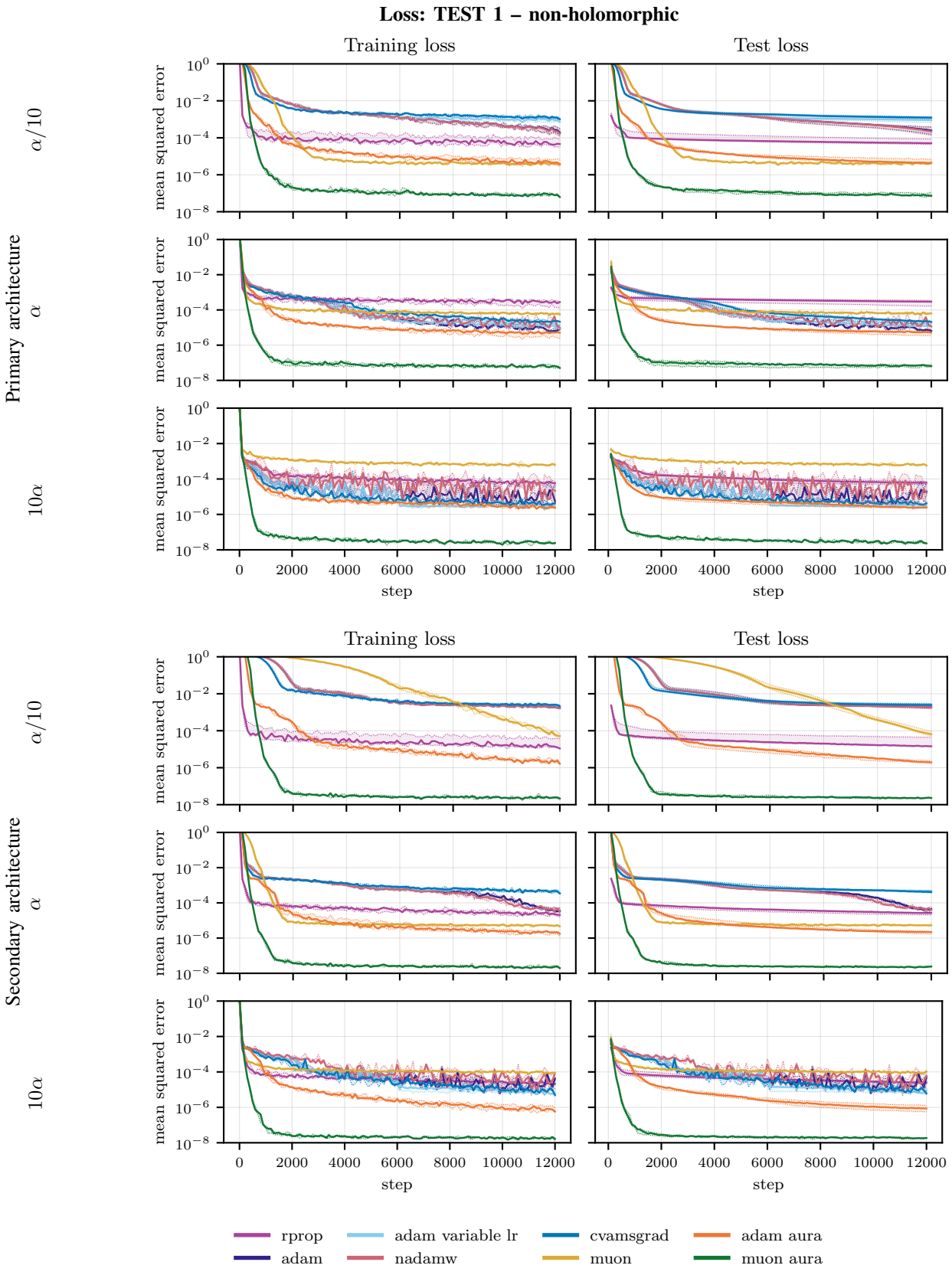


Fig. 1. Training-set and test-set mean squared errors for the Non-holomorphic optimizer benchmark. The two architecture blocks show the network trained at the primary (top) and secondary (bottom) architecture (Table III). Within each block, the rows sweep the base learning rate α across $\alpha/10$, α and 10α from top to bottom. Solid curves show the pointwise median over all 5 seeds; shaded regions span the interquartile range (25th–75th percentile) over seeds, whose edges are also drawn as dotted curves.

TABLE V
β_1/β_2 SENSITIVITY FOR THE NON-HOLOMORPHIC BENCHMARK.

β_1	β_2	RPROP	Adam	Adam (variable LR)	NadamW	CvAMSGrad	Muon	Adam-AURA	Muon-AURA
		$\mathscr{L}_{\min}^{(m)}$ $(\times 10^{-6})$							
0.85	0.99	110^{+49}_{-15}	$4.4^{+0.2}_{-0.4}$	$5.5^{+0.4}_{-0.7}$	$5.2^{+1.9}_{-0.2}$	55^{+37}_{-5}	$41^{+0.2}_{-4}$	$5.8^{+1.1}_{-1.0}$	$\mathbf{0.04^{+0.01}_{-0.001}}$
0.9	0.99	110^{+50}_{-15}	$3.8^{+0.04}_{-0.4}$	$4.9^{+0.03}_{-0.5}$	$5.1^{+0.4}_{-0.8}$	$58^{+45}_{-0.1}$	34^{+2}_{-2}	$2.0^{+0.6}_{-0.3}$	$\mathbf{0.04^{+0.002}_{-0.003}}$
0.95	0.99	110^{+50}_{-15}	$2.6^{+0.4}_{-0.03}$	$4.1^{+0.1}_{-0.4}$	$3.3^{+0.2}_{-0.4}$	151^{+29}_{-37}	$30^{+1}_{-0.1}$	$0.5^{+0.8}_{-0.04}$	$\mathbf{0.03^{+0.003}_{-0.002}}$
0.85	0.999	110^{+50}_{-15}	$5.7^{+0.2}_{-0.4}$	$8.0^{+1.4}_{-0.1}$	$7.5^{+2.2}_{-0.4}$	15^{+2}_{-2}	39^{+1}_{-3}	$5.3^{+2.2}_{-1.0}$	$\mathbf{0.04^{+0.01}_{-0.003}}$
0.9	0.999	128^{+41}_{-24}	$4.9^{+0.1}_{-0.2}$	$7.6^{+1.3}_{-0.2}$	$6.6^{+0.4}_{-0.6}$	16^{+2}_{-2}	$37^{+0.1}_{-3}$	$1.9^{+1.1}_{-0.1}$	$\mathbf{0.04^{+0.01}_{-0.001}}$
0.95	0.999	110^{+50}_{-15}	$4.1^{+0.1}_{-0.1}$	$9.3^{+0.02}_{-1.1}$	$4.4^{+0.3}_{-0.01}$	21^{+2}_{-2}	27^{+2}_{-1}	$0.6^{+0.8}_{-0.1}$	$\mathbf{0.03^{+0.003}_{-0.0002}}$
0.85	0.9999	128^{+41}_{-24}	$9.2^{+0.5}_{-0.3}$	35^{+1}_{-5}	$11^{+2}_{-0.5}$	$6.5^{+1.0}_{-0.3}$	30^{+3}_{-7}	$10^{+0.3}_{-2}$	$\mathbf{0.04^{+0.01}_{-0.003}}$
0.9	0.9999	110^{+50}_{-15}	$8.7^{+0.5}_{-0.4}$	$36^{+0.4}_{-5}$	$9.3^{+0.6}_{-0.5}$	$6.5^{+0.7}_{-0.2}$	20^{+3}_{-7}	$3.1^{+1.2}_{-0.1}$	$\mathbf{0.04^{+0.01}_{-0.001}}$
0.95	0.9999	110^{+50}_{-15}	$10^{+1}_{-0.1}$	40^{+6}_{-2}	$9.7^{+0.5}_{-0.3}$	$7.4^{+0.5}_{-0.4}$	$7.9^{+10.2}_{-0.9}$	$1.2^{+0.8}_{-0.1}$	$\mathbf{0.03^{+0.004}_{-0.001}}$
		$A^{(m)}$							
0.85	0.99	$6.4^{+0.1}_{-0.1}$	$5.6^{+0.1}_{-0.05}$	$5.5^{+0.1}_{-0.1}$	$5.9^{+0.1}_{-0.04}$	$6.6^{+0.1}_{-0.1}$	$6.0^{+0.005}_{-0.01}$	$5.2^{+0.1}_{-0.1}$	$\mathbf{3.1^{+0.04}_{-0.03}}$
0.9	0.99	$6.4^{+0.1}_{-0.1}$	$5.5^{+0.04}_{-0.05}$	$5.4^{+0.04}_{-0.1}$	$5.7^{+0.1}_{-0.04}$	$6.7^{+0.05}_{-0.1}$	$6.0^{+0.01}_{-0.01}$	$4.9^{+0.1}_{-0.1}$	$\mathbf{3.1^{+0.003}_{-0.04}}$
0.95	0.99	$6.4^{+0.1}_{-0.1}$	$5.4^{+0.04}_{-0.1}$	$5.4^{+0.03}_{-0.1}$	$5.4^{+0.04}_{-0.05}$	$6.8^{+0.02}_{-0.1}$	$5.9^{+0.01}_{-0.01}$	$4.4^{+0.2}_{-0.03}$	$\mathbf{3.0^{+0.02}_{-0.02}}$
0.85	0.999	$6.4^{+0.1}_{-0.1}$	$5.8^{+0.1}_{-0.01}$	$5.8^{+0.1}_{-0.01}$	$6.1^{+0.1}_{-0.02}$	$6.0^{+0.05}_{-0.05}$	$6.0^{+0.001}_{-0.01}$	$5.4^{+0.1}_{-0.1}$	$\mathbf{3.1^{+0.03}_{-0.1}}$
0.9	0.999	$6.4^{+0.1}_{-0.1}$	$5.8^{+0.05}_{-0.02}$	$5.8^{+0.1}_{-0.01}$	$5.9^{+0.1}_{-0.02}$	$6.1^{+0.1}_{-0.03}$	$6.0^{+0.005}_{-0.01}$	$5.1^{+0.1}_{-0.1}$	$\mathbf{3.1^{+0.01}_{-0.1}}$
0.95	0.999	$6.4^{+0.1}_{-0.1}$	$5.8^{+0.03}_{-0.03}$	$5.9^{+0.02}_{-0.04}$	$5.8^{+0.03}_{-0.03}$	$6.2^{+0.1}_{-0.02}$	$5.9^{+0.002}_{-0.01}$	$4.6^{+0.2}_{-0.03}$	$\mathbf{3.1^{+0.01}_{-0.03}}$
0.85	0.9999	$6.4^{+0.1}_{-0.1}$	$6.0^{+0.05}_{-0.02}$	$6.2^{+0.1}_{-0.03}$	$6.1^{+0.1}_{-0.01}$	$5.8^{+0.05}_{-0.04}$	$6.0^{+0.003}_{-0.004}$	$5.5^{+0.04}_{-0.1}$	$\mathbf{3.1^{+0.03}_{-0.1}}$
0.9	0.9999	$6.4^{+0.1}_{-0.1}$	$6.0^{+0.05}_{-0.01}$	$6.2^{+0.1}_{-0.01}$	$6.0^{+0.1}_{-0.01}$	$5.8^{+0.04}_{-0.03}$	$6.0^{+0.001}_{-0.01}$	$5.2^{+0.1}_{-0.1}$	$\mathbf{3.1^{+0.01}_{-0.1}}$
0.95	0.9999	$6.4^{+0.1}_{-0.1}$	$6.2^{+0.03}_{-0.03}$	$6.3^{+0.1}_{-0.04}$	$6.1^{+0.02}_{-0.03}$	$5.9^{+0.1}_{-0.01}$	$5.9^{+0.003}_{-0.004}$	$4.8^{+0.1}_{-0.05}$	$\mathbf{3.1^{+0.03}_{-0.03}}$

e) Results – β_1 and β_2 sensitivity: Table V repeats the comparison over a 3×3 grid of the first- and second-moment exponential-decay rates $\beta_1 \in \{0.85, 0.9, 0.95\}$ and $\beta_2 \in \{0.99, 0.999, 0.9999\}$, at the base step size α and the primary architecture, with 3 seeds per pair, and Figure 2 shows the corresponding training- and test-loss curves. The two rates enter Adam, Adam (variable LR), NadamW, CvAMSGrad, and the direction of Adam-AURA, whereas they reach Muon and Muon-AURA only through the Adam-type update of the biases, and RPROP does not use them. The RPROP column nonetheless differs between pairs, for example 110 and 128 in $\mathscr{L}_{\min}^{(m)}$, because each pair is a separate run and we did not set the XLA compiler used by JAX [57] to be fully deterministic on the GPU. Muon-AURA attains the lowest $\mathscr{L}_{\min}^{(m)}$ and $A^{(m)}$ at every pair and is insensitive to both rates, with $A^{(m)}$ between 3.0 and 3.1, whereas the $\mathscr{L}_{\min}^{(m)}$ of Muon, exposed to them in the same way, varies by more than a factor of five. Adam-AURA has the second lowest $A^{(m)}$ at every pair and benefits most from a larger β_1: raising β_1 from 0.85 to 0.95 lowers its $\mathscr{L}_{\min}^{(m)}$ by a factor of about 8 to 12 at each β_2, against at most 1.7 for Adam. This is consistent with the mechanism of AURA, since a longer averaging window makes consecutive Adam directions agree more often, which increases the multiplier; the default value $\beta_1 = 0.9$ of Table II therefore leaves this gain unexploited.

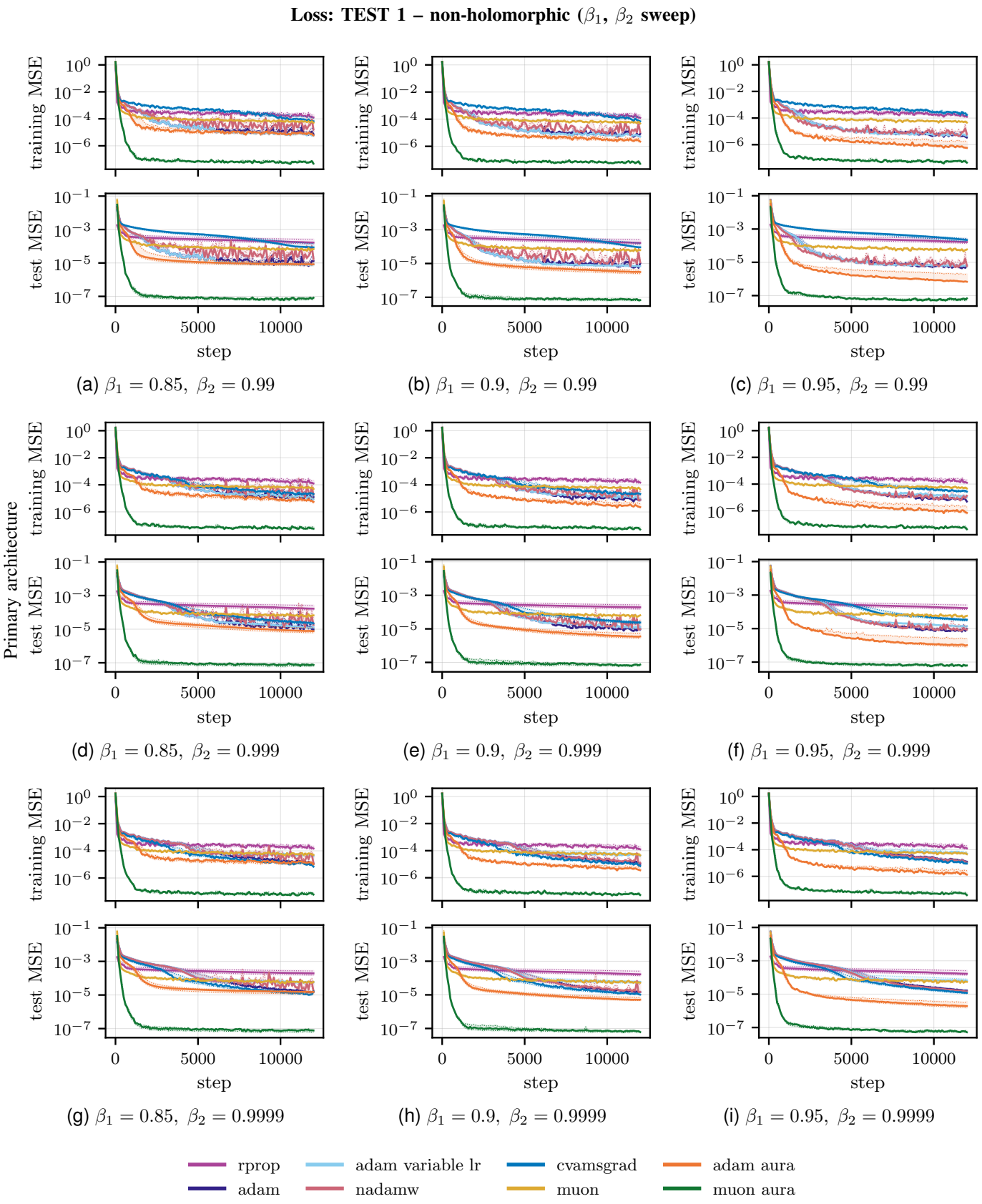


Fig. 2. Training-set and test-set mean squared errors for the Non-holomorphic benchmark over the 3×3 β_1/β_2 grid. Rows fix β_2 (0.99, 0.999, 0.9999); columns fix β_1 (0.85, 0.9, 0.95). The step size is fixed to its baseline, and each panel shows the training error above the test error. Solid curves show the pointwise median over the 3 seeds; shaded regions span the interquartile range (25th–75th percentile) over seeds.

D. TEST 2: univariate scalar complex holomorphic function

a) Rationale: This relatively inexpensive test case allows us to perform multiple training runs. We consider an important class of complex-valued functions: holomorphic functions, which have applications in, e.g., solid mechanics [58], [59]. We investigate the effect of the mini-batch size and show that excessively small mini-batches can constitute a failure mode for AURA.

b) Learning task: Domain sampling and loss construction follow Section IV-C. The activation and target function are specified below.

The domain is the square $\Omega_\zeta = \{\zeta \in \mathbb{C} : |\mathrm{Re}\zeta| \leq 1,\ |\mathrm{Im}\zeta| \leq 1\}$. The target is the entire, multiscale function

$$\begin{aligned} f_{\mathrm{hol}}(\zeta) = \ & \exp((0.39 - 0.22\mathrm{i})\zeta) \\ & + 0.12\exp(3.5\zeta) + 0.03\exp((-2.8 + 3.3\mathrm{i})\zeta) \\ & + 0.002\exp((-5.3 - 4.6\mathrm{i})\zeta) \\ & + 0.0015\exp((5.5 + 5.0\mathrm{i})\zeta) \\ & + 0.015\sin(5.5\zeta) + 0.059\zeta^4 - 0.040\mathrm{i}\zeta^5, \end{aligned} \quad (13)$$

whose oblique complex exponentials produce steep variations near three corners of Ω_ζ, while the sine and polynomial terms contribute oscillatory and algebraic structure.

c) Network architectures: The networks follow the affine–activation composition defined in Section IV-C, with $n_0 = n_L = 1$ and the componentwise entire activation $\varsigma(\zeta) = \exp(\zeta)$. Since complex affine maps and the exponential function are entire, the networks are entire, and hence holomorphic in ζ, by construction.

The complete benchmark configuration, including architectures, activation, dataset sizes, and optimizer hyperparameters, is given in Table VI.

TABLE VI
CONFIGURATION OF THE HOLOMORPHIC OPTIMIZER BENCHMARK.

Setting	Value
Primary architecture	$(1, 32, 32, 32, 32, 1)$, 3,265 trainable parameters
Secondary architecture	$(1, 128, 128, 128, 128, 128, 1)$, 66,433 trainable parameters
Hidden activation	$\exp(\zeta)$
Training / test set	2,500 / 500 points (test: 20%)
Mini-batch size	256 points
Updates / seeds	12,000 / $\{0, \ldots, 4\}$
Arithmetic	`complex64` (32-bit real components)
Initialization	beta-scaled Gaussian-fallback rule for entire activations [58], $\beta_{\mathrm{init}} = 0.5$; zero biases
Base learning rate α (primary)	5×10^{-5}
Secondary-architecture learning rate	5×10^{-6}
Optimizer hyperparameters	default values; see Table II

d) Results – mini-batch size sensitivity: Table VII reports $\mathscr{L}_{\min}^{(m)}$ and $A^{(m)}$ for the mini-batch sizes 32, 128, and 256 at both architectures, together with the number of seeds whose training loss becomes non-finite (NaN); the time ratio is reported only at size 256, the mini-batch size at which the SGD baseline is measured. Where every seed diverges, $A^{(m)}$ is omitted (–), since an area computed over the steps preceding the divergence is not comparable with that of a complete run. Figure 3 shows the corresponding training- and test-loss curves. The update budget and the step size of each architecture are held fixed, so that a smaller mini-batch means fewer passes over the training set, but the same number of updates. At the largest mini-batch, Muon-AURA and Adam-AURA rank first and second in both $\mathscr{L}_{\min}^{(m)}$ and $A^{(m)}$ at both architectures, with the minimum loss of Muon-AURA about 22 to 27 times lower than that of the best baseline; Muon-AURA remains the best method at size 128, where Adam-AURA already diverges in one or two seeds, and Muon-AURA and RPROP in one and two seeds at the secondary architecture. At size 32 the ranking is reversed: both AURA variants diverge in all five seeds at both architectures. RPROP, whose step size also follows the agreement between consecutive steps, is the only baseline that diverges, which suggests that the gradient noise of small mini-batches corrupts this agreement signal; Adam, Adam (variable LR), NadamW, CvAMSGrad, and Muon converge in every run. AURA adds at most 0.13 to the time ratio of its base optimizer.

TABLE VII
MINI-BATCH-SIZE SENSITIVITY FOR THE HOLOMORPHIC BENCHMARK.

		$\mathscr{L}_{\min}^{(m)}$			$A^{(m)}$			Training time (×SGD)	NaN		
Arch.	Optimizer	32	128	256	32	128	256	256	32	128	256
Primary	RPROP	$(3.3^{+10.1}_{-3.1}) \times 10^{-2}$	$(1.5^{+7.5}_{-0.9}) \times 10^{-3}$	$(5.5^{+5.6}_{-1.2}) \times 10^{-4}$	$10.5^{+0.2}_{-0.5}$	$8.3^{+0.6}_{-0.1}$	$7.81^{+0.09}_{-0.04}$	$1.65^{+0.02}_{-0.02}$	1	0	0
	Adam	$\mathbf{(3.3^{+5.6}_{-0.4}) \times 10^{-4}}$	$(2.2^{+2.0}_{-0.04}) \times 10^{-4}$	$(1.3^{+1.4}_{-0.03}) \times 10^{-4}$	$8.86^{+0.06}_{-0.03}$	$8.14^{+0.14}_{-0.04}$	$7.8^{+0.2}_{-0.02}$	$\mathbf{1.55^{+0.03}_{-0.01}}$	0	0	0
	Adam (variable LR)	$(1.2^{+0.2}_{-0.1}) \times 10^{-3}$	$(6.5^{+1.1}_{-1.9}) \times 10^{-4}$	$(4.6^{+3.1}_{-3.1}) \times 10^{-4}$	$8.92^{+0.17}_{-0.03}$	$8.1^{+0.2}_{-0.02}$	$7.8^{+0.3}_{-0.2}$	$1.57^{+0.03}_{-0.03}$	0	0	0
	NadamW	$(3.4^{+1.4}_{-1.4}) \times 10^{-4}$	$(1.4^{+0.9}_{-0.3}) \times 10^{-4}$	$(1.2^{+0.7}_{-0.1}) \times 10^{-4}$	$8.80^{+0.05}_{-0.06}$	$8.0^{+0.1}_{-0.1}$	$7.7^{+0.2}_{-0.01}$	$1.58^{+0.08}_{-0.004}$	0	0	0
	CvAMSGrad	$(6.9^{+5.9}_{-1.3}) \times 10^{-4}$	$(6.8^{+0.6}_{-3.3}) \times 10^{-4}$	$(6.1^{+1.2}_{-2.9}) \times 10^{-4}$	$\mathbf{8.73^{+0.08}_{-0.001}}$	$8.1^{+0.2}_{-0.1}$	$7.9^{+0.2}_{-0.1}$	$1.63^{+0.04}_{-0.06}$	0	0	0
	Muon	$(7.8^{+4.3}_{-0.4}) \times 10^{-4}$	$(3.0^{+0.1}_{-0.1}) \times 10^{-4}$	$(2.1^{+0.3}_{-0.3}) \times 10^{-4}$	$8.76^{+0.14}_{-0.001}$	$8.06^{+0.11}_{-0.03}$	$7.80^{+0.09}_{-0.01}$	$1.88^{+0.03}_{-0.01}$	0	0	0
	Adam-AURA	$(4.0^{+1.1}_{-2.3}) \times 10^{-2}$	$(7.1^{+12.2}_{-0.8}) \times 10^{-5}$	$(5.2^{+0.3}_{-0.8}) \times 10^{-5}$	–	$7.7^{+0.1}_{-0.2}$	$6.75^{+0.05}_{-0.03}$	$1.68^{+0.04}_{-0.09}$	5	2	0
	Muon-AURA	$(2.1^{+0.4}_{-1.3}) \times 10^{-1}$	$\mathbf{(3.3^{+0.9}_{-0.3}) \times 10^{-6}}$	$\mathbf{(5.3^{+2.3}_{-0.4}) \times 10^{-6}}$	–	$\mathbf{5.9^{+0.6}_{-0.003}}$	$\mathbf{5.72^{+0.06}_{-0.02}}$	$1.90^{+0.01}_{-0.11}$	5	0	0
Secondary	RPROP	$(1.3^{+0.2}_{-0.2}) \times 10^{0}$	$(4.5^{+214637.2}_{-0.4}) \times 10^{-5}$	$(7.9^{+1.5}_{-0.9}) \times 10^{-5}$	–	$7^{+4}_{-0.1}$	$6.97^{+0.02}_{-0.01}$	$1.56^{+0.10}_{-0.03}$	5	2	0
	Adam	$(9.0^{+29.4}_{-4.5}) \times 10^{-4}$	$(1.8^{+9.3}_{-0.1}) \times 10^{-4}$	$(1.2^{+2.4}_{-0.2}) \times 10^{-4}$	$9.4^{+0.02}_{-0.4}$	$8.5^{+0.3}_{-0.1}$	$8.1^{+0.4}_{-0.2}$	$\mathbf{1.50^{+0.04}_{-0.03}}$	0	0	0
	Adam (variable LR)	$(2.0^{+2.0}_{-1.6}) \times 10^{-2}$	$(1.3^{+4.5}_{-0.5}) \times 10^{-3}$	$(2.3^{+42.5}_{-0.02}) \times 10^{-4}$	$9.6^{+0.4}_{-0.4}$	$8.7^{+0.3}_{-0.2}$	$8.1^{+0.6}_{-0.2}$	$1.52^{+0.03}_{-0.02}$	0	0	0
	NadamW	$(4.2^{+37.8}_{-1.6}) \times 10^{-4}$	$(9.4^{+59.2}_{-2.2}) \times 10^{-5}$	$(1.1^{+1.5}_{-0.4}) \times 10^{-4}$	$9.4^{+0.02}_{-0.4}$	$8.3^{+0.3}_{-0.2}$	$8.0^{+0.4}_{-0.3}$	$1.51^{+0.06}_{-0.005}$	0	0	0
	CvAMSGrad	$(1.1^{+3.4}_{-0.02}) \times 10^{-3}$	$(5.5^{+20.5}_{-3.8}) \times 10^{-4}$	$(2.6^{+16.6}_{-0.1}) \times 10^{-4}$	$9.2^{+0.1}_{-0.2}$	$8.3^{+0.3}_{-0.03}$	$7.9^{+0.5}_{-0.01}$	$1.53^{+0.04}_{-0.01}$	0	0	0
	Muon	$\mathbf{(8.5^{+0.5}_{-0.8}) \times 10^{-5}}$	$(6.7^{+1.5}_{-2.4}) \times 10^{-5}$	$(4.2^{+1.3}_{-0.6}) \times 10^{-5}$	$\mathbf{8.8^{+0.3}_{-0.04}}$	$8.4^{+0.2}_{-0.1}$	$8.1^{+0.1}_{-0.1}$	$2.09^{+0.02}_{-0.0004}$	0	0	0
	Adam-AURA	$(4.2^{+1.7}_{-3.5}) \times 10^{-2}$	$(3.5^{+4.3}_{-0.3}) \times 10^{-5}$	$(1.9^{+0.6}_{-0.1}) \times 10^{-5}$	–	$7.5^{+0.5}_{-0.2}$	$6.67^{+0.06}_{-0.11}$	$1.60^{+0.06}_{-0.06}$	5	1	0
	Muon-AURA	$(1.8^{+0.4}_{-1.1}) \times 10^{-1}$	$\mathbf{(7.4^{+20.3}_{-4.8}) \times 10^{-6}}$	$\mathbf{(1.6^{+0.1}_{-0.1}) \times 10^{-6}}$	–	$\mathbf{6.7^{+0.4}_{-0.5}}$	$\mathbf{5.20^{+0.0002}_{-0.05}}$	$2.03^{+0.02}_{-0.02}$	5	1	0

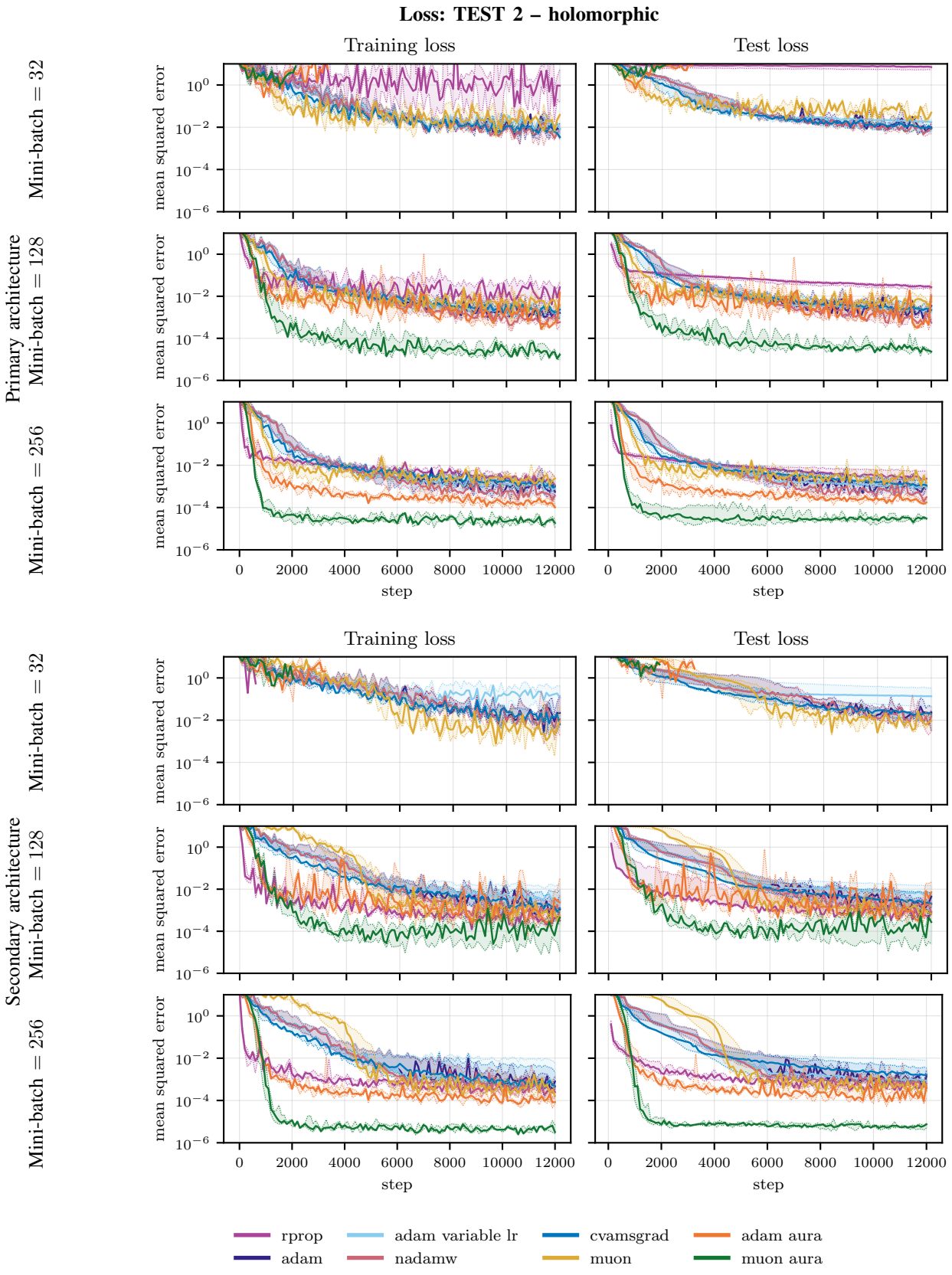


Fig. 3. Training-set (left panel of each row) and test-set (right panel) mean squared errors for the Holomorphic optimizer benchmark. The two architecture blocks show the same feedforward network trained independently at two architectures (Table VI): the primary architecture $(1, 32, 32, 32, 32, 1)$ (3,265 parameters) at the top, and the wider and deeper secondary architecture $(1, 128, 128, 128, 128, 128, 1)$ (66,433 parameters), trained at a ten-times-smaller step size, at the bottom. Within each architecture block, the rows sweep the mini-batch size across 32, 128 and 256 points from top to bottom at a fixed 12,000-update budget, so a smaller mini-batch means fewer epochs over the training set, not fewer optimizer updates. Solid curves show the pointwise median over all 5 seeds; shaded regions span the interquartile range (25th–75th percentile) over seeds, whose edges are also drawn as dotted curves.

E. TEST 3: multivariate complex $\mathbb{C}^4$ function

a) Rationale: This test uses a classical benchmark for complex-valued networks with four complex inputs [36], [52], [60]–[63]. We investigate the sensitivity of the optimizers to the floating-point precision, training a single architecture in single and double precision, and repeat the step-size sweep of Section IV-C.

b) Learning task: The target is the multivariate benchmark function considered in Section 5.1 of [36]:

$$f_{\mathrm{mc4}}(\zeta_1,\zeta_2,\zeta_3,\zeta_4)=\frac{1}{1.5}\left(\frac{\zeta_2^2}{\zeta_1}+\zeta_3+10\,\zeta_1\zeta_4\right),\qquad \boldsymbol{\zeta}\in\Omega_\zeta. \tag{14}$$

The training and test sets are drawn as in (10)–(11), with

$$\Omega_\zeta=\left\{\boldsymbol{\zeta}\in\mathbb{C}^4:\mathrm{Re}\zeta_k,\mathrm{Im}\zeta_k\in[0.5,1],\ k=1,\dots,4\right\}. \tag{15}$$

This domain keeps ζ_1 away from zero, ensuring that the target is well defined and holomorphic on a neighborhood of Ω_ζ. The loss is the mean squared error (12), evaluated at the vector-valued inputs $\boldsymbol{\zeta}$.

c) Network architecture: A single network is used, following the affine–activation composition defined in Section IV-C with $n_0=4$ inputs, the components of $\boldsymbol{\zeta}=(\zeta_1,\zeta_2,\zeta_3,\zeta_4)\in\mathbb{C}^4$, and $n_L=1$ output. The bounded activation

$$\varsigma(\zeta)=\tanh(\mathrm{Re}\zeta)+\mathrm{i}\tanh(\mathrm{Im}\zeta),$$

is applied componentwise in the hidden layers, so the network is generally non-holomorphic, although the target is holomorphic. Here, unlike in Section IV-C, we did not use the SiLU function, in order to increase the variability of the test cases.

The complete benchmark configuration, including the architecture, activation, dataset sizes, and the swept step sizes and precisions, is given in Table VIII.

TABLE VIII
CONFIGURATION OF THE MULTIVARIATE C4 OPTIMIZER BENCHMARK.

Setting	Value
Network architecture	$(4,128,128,128,128,128,1)$, 66,817 trainable parameters
Hidden activation	$\tanh(\mathrm{Re}\zeta)+\mathrm{i}\tanh(\mathrm{Im}\zeta)$ (componentwise across the four inputs)
Training / test set	1,296 / 259 points (test: 20%)
Mini-batch size	128 points
Updates / seeds	12,000 / $\{0,\dots,4\}$
Arithmetic	`complex64` and `complex128` (single- and double-precision real components; both swept)
Initialization	Glorot/Xavier normal rule [56], applied separately to the real and imaginary parts; zero biases
Base learning rate α	1×10^{-4}, swept over $\alpha/10$, α and 10α
Optimizer hyperparameters	default values; see Table II

d) Results – α and precision sensitivity: Table IX reports the three metrics for the step sizes $\alpha/10$, α, and 10α in single (`complex64`) and double (`complex128`) precision, and Figure 4 shows the corresponding training- and test-loss curves. The precision has little influence: double precision changes $A^{(m)}$ by at most about 0.1 for every method and step size other than Muon-AURA, and only Muon-AURA improves by more than 0.1 at every step size, by 0.11 to 0.28. The step size matters far more: over its 100-fold range, $\mathscr{L}_{\min}^{(m)}$ varies by more than an order of magnitude for Adam, its variants, and Muon, but by at most about a factor of two for Adam-AURA; RPROP is similarly insensitive, but its minimum loss is about thirty times that of Adam-AURA. Muon-AURA attains the lowest $\mathscr{L}_{\min}^{(m)}$ and $A^{(m)}$ in all six settings, with a margin of about 2 in $A^{(m)}$ over the best baseline, that is, a geometric-mean training loss about two orders of magnitude lower, and Adam-AURA ranks second in $A^{(m)}$ in all six settings. AURA adds at most 0.15 to the time ratio of its base optimizer.

TABLE IX
STEP-SIZE AND PRECISION SENSITIVITY FOR THE MULTIVARIATE C4 BENCHMARK.

Prec.	Optimizer	$\mathscr{L}_{\min}^{(m)}$ ($\times 10^{-4}$)			$A^{(m)}$			Training time ($\times$SGD)
		$\alpha/10$	α	10α	$\alpha/10$	α	10α	α
Single	RPROP	38^{+3}_{-4}	36^{+2}_{-1}	31^{+5}_{-2}	$8.04^{+0.01}_{-0.07}$	$7.97^{+0.15}_{-0.01}$	$7.927^{+0.009}_{-0.007}$	$1.55^{+0.05}_{-0.02}$
	Adam	361^{+19}_{-14}	$4.1^{+0.2}_{-0.2}$	$2.2^{+0.04}_{-0.1}$	$9.50^{+0.002}_{-0.05}$	$8.02^{+0.02}_{-0.05}$	$7.85^{+0.02}_{-0.01}$	$1.55^{+0.0004}_{-0.11}$
	Adam (variable LR)	997^{+49}_{-143}	$12^{+0.5}_{-0.5}$	$1.7^{+0.1}_{-0.1}$	$9.68^{+0.01}_{-0.07}$	$8.15^{+0.005}_{-0.10}$	$7.40^{+0.04}_{-0.002}$	$1.53^{+0.03}_{-0.01}$
	NadamW	321^{+35}_{-7}	$7.4^{+0.3}_{-0.4}$	$8.9^{+0.6}_{-3.3}$	$9.50^{+0.001}_{-0.05}$	$8.23^{+0.01}_{-0.04}$	$8.43^{+0.01}_{-0.03}$	$\mathbf{1.51^{+0.03}_{-0.01}}$
	CvAMSGrad	1821^{+3}_{-508}	$18^{+0.1}_{-5}$	$2.2^{+0.2}_{-0.01}$	$9.84^{+0.02}_{-0.06}$	$8.25^{+0.03}_{-0.13}$	$7.34^{+0.03}_{-0.03}$	$1.6^{+0.1}_{-0.1}$
	Muon	$0.8^{+0.01}_{-0.1}$	$27^{+0.5}_{-1}$	$12^{+0.1}_{-0.4}$	$8.35^{+0.03}_{-0.09}$	$7.75^{+0.01}_{-0.02}$	$7.818^{+0.017}_{-0.003}$	$2.128^{+0.013}_{-0.006}$
	Adam-AURA	$1.2^{+0.03}_{-0.04}$	$0.8^{+0.1}_{-0.1}$	$0.9^{+0.2}_{-0.01}$	$7.10^{+0.06}_{-0.01}$	$6.74^{+0.06}_{-0.004}$	$6.68^{+0.09}_{-0.02}$	$1.62^{+0.07}_{-0.002}$
	Muon-AURA	$\mathbf{0.1^{+0.01}_{-0.003}}$	$\mathbf{0.1^{+0.01}_{-0.001}}$	$\mathbf{0.1^{+0.005}_{-0.002}}$	$\mathbf{5.75^{+0.01}_{-0.03}}$	$\mathbf{5.64^{+0.01}_{-0.06}}$	$\mathbf{5.34^{+0.02}_{-0.03}}$	$2.07^{+0.02}_{-0.02}$
Double	RPROP	41^{+23}_{-10}	27^{+1}_{-1}	32^{+1}_{-5}	$8.1^{+0.2}_{-0.2}$	$7.85^{+0.04}_{-0.01}$	$7.903^{+0.004}_{-0.005}$	$\mathbf{1.55^{+0.06}_{-0.01}}$
	Adam	332^{+42}_{-10}	$4.1^{+0.1}_{-0.1}$	$2.0^{+0.1}_{-0.1}$	$9.47^{+0.04}_{-0.01}$	$8.01^{+0.01}_{-0.05}$	$7.860^{+0.006}_{-0.010}$	$1.62^{+0.02}_{-0.02}$
	Adam (variable LR)	1010^{+6}_{-111}	$10^{+1}_{-0.5}$	$1.7^{+0.2}_{-0.1}$	$9.65^{+0.05}_{-0.02}$	$8.09^{+0.04}_{-0.06}$	$7.411^{+0.002}_{-0.017}$	$1.61^{+0.07}_{-0.07}$
	NadamW	291^{+46}_{-11}	$7.0^{+1.1}_{-0.3}$	$7.7^{+1.8}_{-1.4}$	$9.46^{+0.05}_{-0.01}$	$8.22^{+0.005}_{-0.03}$	$8.426^{+0.003}_{-0.010}$	$1.644^{+0.006}_{-0.007}$
	CvAMSGrad	1540^{+179}_{-170}	$16^{+4}_{-0.4}$	$2.4^{+0.3}_{-0.1}$	$9.79^{+0.05}_{-0.002}$	$8.16^{+0.08}_{-0.05}$	$7.35^{+0.04}_{-0.02}$	$1.55^{+0.10}_{-0.05}$
	Muon	$1.2^{+0.1}_{-0.1}$	$30^{+0.4}_{-1}$	$14^{+0.4}_{-1}$	$8.4^{+0.2}_{-0.1}$	$7.781^{+0.016}_{-0.0001}$	$7.86^{+0.02}_{-0.05}$	$2.47^{+0.04}_{-0.02}$
	Adam-AURA	$1.2^{+0.2}_{-0.02}$	$0.9^{+0.1}_{-0.1}$	$0.9^{+0.2}_{-0.1}$	$7.14^{+0.02}_{-0.02}$	$6.78^{+0.03}_{-0.02}$	$6.69^{+0.05}_{-0.07}$	$1.76^{+0.06}_{-0.08}$
	Muon-AURA	$\mathbf{0.1^{+0.01}_{-0.002}}$	$\mathbf{0.03^{+0.002}_{-0.002}}$	$\mathbf{0.03^{+0.002}_{-0.001}}$	$\mathbf{5.64^{+0.05}_{-0.07}}$	$\mathbf{5.36^{+0.03}_{-0.02}}$	$\mathbf{5.13^{+0.02}_{-0.02}}$	$2.44^{+0.07}_{-0.04}$

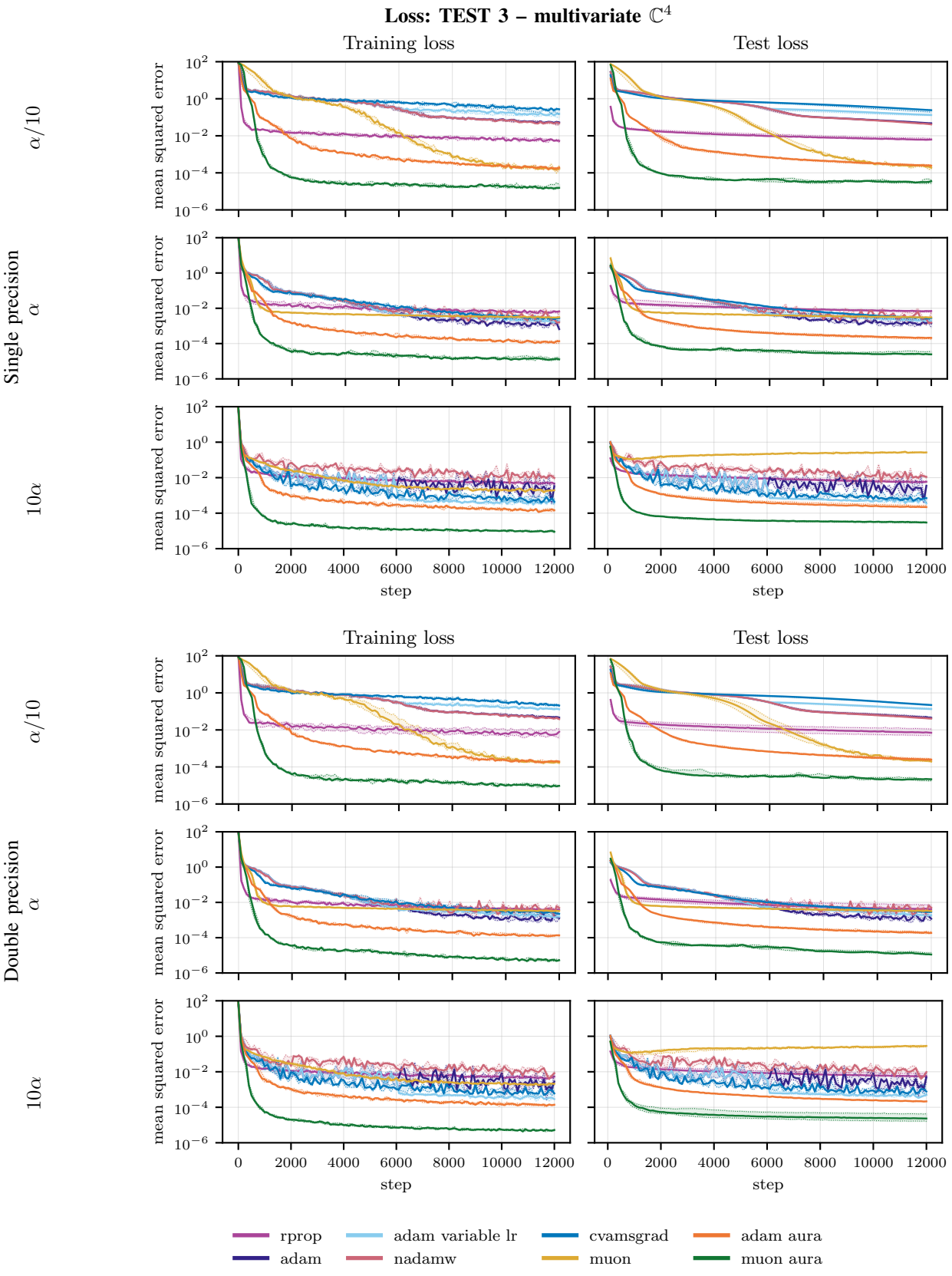


Fig. 4. Training-set and test-set mean squared errors for the Multivariate C4 optimizer benchmark, on the single benchmark architecture $(4, 128, 128, 128, 128, 128, 1)$ (66,817 parameters; Table VIII). The two precision blocks show the same network trained in single-precision (`complex64`, top) and double-precision (`complex128`, bottom) arithmetic. Within each precision block, the rows sweep the base step size $\alpha = 1 \times 10^{-4}$ across $\alpha/10$, α and 10α from top to bottom. Solid curves show the pointwise median over all 5 seeds; shaded regions span the interquartile range (25th–75th percentile) over seeds, whose edges are also drawn as dotted curves.

F. TEST 4: PINN

a) Rationale: This test evaluates the optimizers on a physics-informed learning task while retaining a fully connected holomorphic architecture. In addition, the hyperparameters of every optimizer are tuned under a limited budget, as recommended in [14], so that the methods are compared at, or close to, their best performance and no baseline is penalized by an unfavorable default.

b) Learning task: The ultimate goal is to solve the two-dimensional linear elasticity problem for a homogeneous material. We consider the geometry and boundary conditions of the plate-with-a-circular-hole benchmark presented in [58]. The computational domain is the upper-left quadrant

$$\Omega_\zeta = \{\zeta = x + \mathrm{i}y : \, -a < x < 0, \; 0 < y < a, \; x^2 + y^2 > r^2\},$$

with $a = 2.5$ m and $r = 1$ m. Traction conditions are imposed on the outer edges and hole arc, while symmetry conditions are imposed on the cut edges; see Section 4.1.2 of [58] for details.

Two holomorphic neural networks, $\varphi_\mathcal{W} : \mathbb{C} \to \mathbb{C}$ and $\psi_\mathcal{W} : \mathbb{C} \to \mathbb{C}$, take the spatial coordinate $\zeta \in \Omega_\zeta$ as input and approximate the Kolosov–Muskhelishvili potentials φ and ψ, respectively. Under plane-strain conditions with $\lambda = \mu = 1$ MPa, the Kolosov constant is $\kappa = (\lambda + 3\mu)/(\lambda + \mu) = 2$, and stresses and displacements follow from

$$\begin{aligned} \sigma_{xx} &= \mathrm{Re}(2\varphi' - \overline{\zeta}\varphi'' - \psi'), \\ \sigma_{yy} &= \mathrm{Re}(2\varphi' + \overline{\zeta}\varphi'' + \psi'), \\ \sigma_{xy} &= \mathrm{Im}(\overline{\zeta}\varphi'' + \psi'), \end{aligned} \tag{16}$$

$$u_x + \mathrm{i}u_y = \frac{1}{2\mu}\big(\kappa\varphi - \zeta\overline{\varphi'} - \overline{\psi}\big). \tag{17}$$

Primes denote differentiation with respect to ζ, evaluated by holomorphic automatic differentiation. This representation satisfies the governing elasticity equations by construction, so training minimizes only the boundary-condition residuals:

$$\begin{aligned} \mathscr{L}_{\mathrm{PIHNN}}(\mathcal{W}) = \frac{\alpha_N}{|\mathcal{Z}_N|} \sum_{\zeta \in \mathcal{Z}_N} \|\boldsymbol{\sigma}_\mathcal{W}(\zeta)\mathbf{n} - \mathbf{t}_0(\zeta)\|^2 \\ + \frac{\alpha_S}{|\mathcal{Z}_S|} \sum_{\zeta \in \mathcal{Z}_S} \big(\sigma_{xy,\mathcal{W}}(\zeta)^2 + |\boldsymbol{u}_\mathcal{W}(\zeta) \cdot \mathbf{n}|^2\big). \end{aligned} \tag{18}$$

Here, $\mathcal{Z}_N$ and $\mathcal{Z}_S$ contain the points on the traction and symmetry boundaries, respectively, $\mathbf{n}$ is the outward unit normal, $\mathbf{t}_0$ is the prescribed traction, and α_N, α_S are the corresponding boundary-length fractions defined in [58]. The boundary points remain fixed throughout training, and every update uses the full training set.

c) Network architecture: We use two fully connected networks, $\varphi_\mathcal{W}$ and $\psi_\mathcal{W}$, each with

$$(n_0, n_1, n_2, n_3, n_4) = (1, 64, 64, 64, 1).$$

The three hidden layers use the componentwise activation $\varsigma(\zeta) = \exp(\zeta)$, and the output layer is affine. Both potentials are entire functions of ζ. The networks are wider than in [58], which uses 10 units per hidden layer. A larger network makes the benchmark more meaningful for the optimizers. The wider networks require a smaller learning rate, 10^{-3} instead of 10^{-2}. Adam (variable LR) uses the learning-rate schedule of [58].

The complete benchmark configuration is given in Table X.

TABLE X
CONFIGURATION OF THE PINN OPTIMIZER BENCHMARK.

Setting	Value
Architecture (each of φ, ψ)	$(1, 64, 64, 64, 1)$, 17,026 trainable parameters in total
Hidden activation	$\exp(\zeta)$
Training / test set	200 / 20 boundary points (full batch, no mini-batching)
Updates / seeds	6,000 / $\{0, \ldots, 4\}$ (weight initialization only, boundary sampling fixed)
Arithmetic	`complex64` (32-bit real components)
Initialization	initialization rule defined in [58] (their Algorithm 1), $\beta_{\mathrm{init}} = 0.5$; zero biases
Base learning rate α	1×10^{-3}, starting point of the search; tuned values in Table XI
Optimizer hyperparameters	tuned per method (Table XI), starting from the default values of Table II

d) Hyperparameter optimization: The learning rate and the principal hyperparameters of every optimizer are searched jointly with the multivariate tree-structured Parzen estimator (TPE) [64] of Optuna [65]; Table XI lists the search space and the selected values, and every other hyperparameter keeps the value of Table II. Every searched quantity is sampled on a logarithmic scale except χ_a; χ_o and Ψ_a are obtained from the sampled gap $\chi_\mathrm{a} - \chi_\mathrm{o}$ and ratio $\Psi_\mathrm{a}/\Psi_\mathrm{o}$, which preserve the orderings of Table I. Each method receives the same budget of 50 trials: the first evaluates the starting point, namely the values of Table II with $\alpha = 10^{-3}$, sampling is random until ten trials are complete, and TPE proposes the remaining ones. A trial trains the networks from three weight-initialization seeds, $\{100, 101, 102\}$, disjoint from the evaluation seeds of Table X,

TABLE XI
HYPERPARAMETER SEARCH OF THE PINN BENCHMARK.

Optimizer	Search space	Selected values
RPROP	$\alpha \in [10^{-4}, 10^{-1}]$, $1-\eta_- \in [0.01, 0.6]$, $\eta_+ - 1 \in [0.005, 0.5]$	$\alpha = 9.727 \times 10^{-3}$, $\eta_- = 0.7131$, $\eta_+ = 1.179$
Adam	$\alpha \in [10^{-4}, 10^{-1}]$, $1-\beta_1 \in [10^{-3}, 0.3]$, $1-\beta_2 \in [10^{-4}, 0.1]$	$\alpha = 1.888 \times 10^{-3}$, $\beta_1 = 0.9843$, $\beta_2 = 0.9998$
Adam (variable LR)	as Adam	$\alpha = 3.968 \times 10^{-3}$, $\beta_1 = 0.9851$, $\beta_2 = 0.9229$
NadamW	as Adam, and weight decay $\in [10^{-5}, 0.1]$	$\alpha = 1.249 \times 10^{-3}$, $\beta_1 = 0.9969$, $\beta_2 = 0.9795$, weight decay 3.437×10^{-5}
CvAMSGrad	as Adam	$\alpha = 3.403 \times 10^{-3}$, $\beta_1 = 0.9818$, $\beta_2 = 0.998$
Muon	$\alpha \in [10^{-4}, 10^{-1}]$, $1-\beta \in [10^{-3}, 0.3]$	$\alpha = 2.898 \times 10^{-4}$, $\beta = 0.9962$
Adam-AURA	$\alpha \in [10^{-4}, 10^{-1}]$, $1-\beta_\zeta \in [0.01, 0.6]$, $\chi_\mathrm{a} \in [0.5, 0.98]$, $\chi_\mathrm{a} - \chi_\mathrm{o} \in [0.02, 1.2]$, $\Psi_\mathrm{o} \in [0.01, 0.9]$, $\Psi_\mathrm{a}/\Psi_\mathrm{o} \in [0.005, 0.9]$; β_1, β_2 as selected for Adam	$\alpha = 1.171 \times 10^{-3}$, $\beta_\zeta = 0.973$, $\chi_\mathrm{a} = 0.7383$, $\chi_\mathrm{o} = -0.4217$, $\Psi_\mathrm{a} = 0.02422$, $\Psi_\mathrm{o} = 0.03051$
Muon-AURA	as Muon and as Adam-AURA	$\alpha = 1.816 \times 10^{-3}$, $\beta = 0.9732$, $\beta_\zeta = 0.8016$, $\chi_\mathrm{a} = 0.677$, $\chi_\mathrm{o} = -0.0416$, $\Psi_\mathrm{a} = 0.0167$, $\Psi_\mathrm{o} = 0.1191$

and is scored by the mean of $A^{(m,s)}$ (7) over them; a trial whose loss becomes non-finite on any seed receives a score of at least $C = 10$, the area of a loss held at 1, and is excluded from the selection. The admissible trial with the lowest score is selected and evaluated on the five seeds of Table X as in the other cases, without an intermediate confirmation stage; the search costs $8 \times 50 \times 3 = 1{,}200$ training runs. Adam-AURA is searched after Adam, with β_1 and β_2 fixed at the values selected for Adam, so that only α and the five gates β_ζ, χ_a, χ_o, Ψ_a, and Ψ_o are searched. This exploits the plug-in property of AURA and isolates the effect of its gates on a tuned base optimizer; counted as a whole, Adam-AURA thus receives twice the budget of the other methods, but the coupling between the Adam and the AURA hyperparameters is left unexplored. Muon-AURA is instead searched independently.

e) Results: Table XII reports the three metrics at the tuned hyperparameters, together with the number of seeds whose training loss becomes non-finite (NaN), and Figure 5 shows the training- and test-loss curves at the tuned (bottom) hyperparameters. For completeness, the same figure also shows the results at the non-tuned hyperparameters, that is, those in Table II, to better illustrate the effect of the hyperparameter optimization. Tuning benefits every method, the baselines more than the AURA variants: it lowers $A^{(m)}$ by 0.2 to 1.8 for the baselines and by 0.5 and 0.8 for Adam-AURA and Muon-AURA, respectively. The overall conclusion is unchanged: Muon-AURA and Adam-AURA attain the two lowest values of both $\mathscr{L}_{\min}^{(m)}$ and $A^{(m)}$; the minimum loss of Muon-AURA is about 3.5 times below that of the best baseline, NadamW, and its margin of 1.3 in $A^{(m)}$ corresponds to a geometric-mean training loss about 20 times lower; the minimum loss of Adam-AURA is about 1.7 times below that of NadamW and its margin of 0.8 in $A^{(m)}$ to a loss about six times lower. The largest effect of AURA is on Muon: alone, it reaches the highest minimum loss of all methods, and its combination with AURA lowers it by more than two orders of magnitude. At their tuned values, which include a step size four to ten times larger than the starting value of the search, RPROP and Adam (variable LR) diverge on one of the five evaluation seeds, although neither diverged on the three search seeds; every other method converges on every seed. In this full-batch setting, the methods whose step sizes follow the agreement between consecutive steps, and which diverged at small mini-batches in Section IV-D, rank first and second in $A^{(m)}$ (AURA) or within 0.2 of the best baseline (RPROP), consistent with the interpretation given there.

TABLE XII
TRAINING LOSS AND TRAINING TIME FOR THE PINN BENCHMARK AT THE TUNED HYPERPARAMETERS OF TABLE XI.

Optimizer	$\mathscr{L}_{\min}^{(m)}$	$A^{(m)}$	Training time (×SGD)	NaN
RPROP	$(4.3^{+2.5}_{-2.0}) \times 10^{-5}$	$5.9^{+0.1}_{-0.2}$	$1.2^{+0.1}_{-0.1}$	1
Adam	$(1.0^{+0.02}_{-0.1}) \times 10^{-5}$	$6.26^{+0.01}_{-0.01}$	$\mathbf{1.21^{+0.03}_{-0.003}}$	0
Adam (variable LR)	$(1.8^{+0.5}_{-0.4}) \times 10^{-5}$	$6.3^{+0.1}_{-0.2}$	$1.254^{+0.003}_{-0.009}$	1
NadamW	$(2.2^{+0.2}_{-0.03}) \times 10^{-6}$	$5.76^{+0.02}_{-0.02}$	$1.249^{+0.001}_{-0.006}$	0
CvAMSGrad	$(5.7^{+1.5}_{-1.2}) \times 10^{-6}$	$6.0^{+0.2}_{-0.1}$	$1.37^{+0.02}_{-0.02}$	0
Muon	$(8.0^{+0.9}_{-1.1}) \times 10^{-5}$	$6.60^{+0.05}_{-0.005}$	$1.47^{+0.0004}_{-0.05}$	0
Adam-AURA	$(1.3^{+0.4}_{-0.1}) \times 10^{-6}$	$4.96^{+0.05}_{-0.05}$	$1.39^{+0.04}_{-0.03}$	0
Muon-AURA	$\mathbf{(6.3^{+1.6}_{-0.7}) \times 10^{-7}}$	$\mathbf{4.46^{+0.01}_{-0.12}}$	$2.09^{+0.01}_{-0.01}$	0

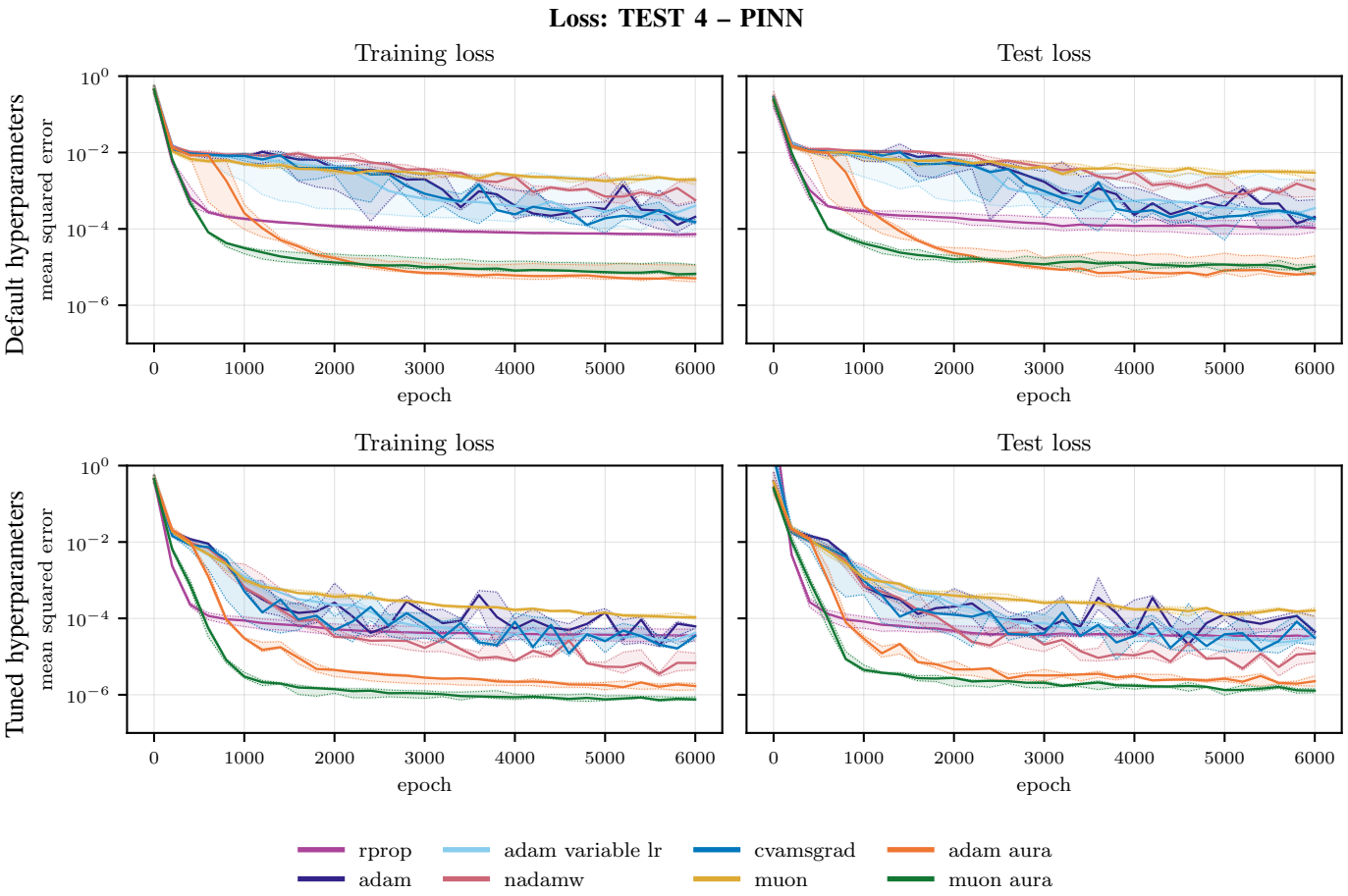


Fig. 5. Training-set and test-set mean squared errors (boundary loss (18)) for the PINN optimizer benchmark. The two rows show the same network trained at the default hyperparameters of Table II with $\alpha = 10^{-3}$ (top) and at the tuned hyperparameters of Table XI (bottom). Solid curves show the pointwise median over all 5 seeds; shaded regions span the interquartile range (25th–75th percentile) over seeds, whose edges are also drawn as dotted curves.

V. Conclusions

We have proposed AURA, a per-parameter step-size multiplier for training CVNNs. AURA acts on top of any first-order optimizer and leaves its update direction unchanged. It compares consecutive update directions of each complex parameter through a Dice-like measure, whose real part quantifies their alignment and whose imaginary part their signed rotation. The step is enlarged when the directions agree in length, alignment, and sense of rotation, and reduced when they do not. No additional gradient evaluation is required. We do not claim that the results achieved are optimal. Rather, the presented findings indicate that AURA can be adopted to improve the results of its base optimizer.

We combined AURA with Adam and Muon and compared the two variants with RPROP, Adam, NadamW, CvAMSGrad, and Muon on four test cases with fully connected CVNNs, from the approximation of scalar complex functions to a physics-informed problem. The hyperparameters of AURA were held fixed across cases; in Section IV-F, every optimizer was tuned under the same budget. Muon-AURA attained the lowest minimum loss and the lowest area under the log learning curve in every setting with mini-batches of at least 128 points and in the full-batch case, at both fixed and tuned hyperparameters. Adam-AURA ranked second in the area metric in most settings. AURA also reduced the sensitivity to the step size: over a 100-fold range of α, the minimum loss of Adam-AURA varied by less than an order of magnitude, against more than two for Adam. Tuning benefited the baselines more than the AURA variants, which nonetheless remained the two best methods. In the mini-batch cases, AURA added at most 0.15 to the time ratio of its base optimizer.

From the obtained results, we can conclude that AURA fails when the gradient is noisy. At the smallest mini-batch of Section IV-D, both variants diverged in every seed, together with RPROP. The cause lies in the agreement signal itself: AURA inherits no estimate of the gradient variance and measures the agreement between noisy directions. The full-batch case of Section IV-F, where the two variants rank first and second, is consistent with this interpretation.

AURA has three further limitations. It stores five real numbers per parameter in addition to the state of the base optimizer, against the three of Adam. It adds a few elementwise operations per step. It introduces ten hyperparameters, of which the decay rate β_ζ and the four thresholds are the least intuitive to set. On the other hand, the relatively high number of hyperparameters can be an advantage as it represents a

margin for adaptation.

Several developments follow from these results. The failure mode calls for a signal-to-noise estimate in the gate, so that the multiplier is not increased on the agreement of noisy directions. The current real-valued step size can be naturally extended to a complex step size, which rotates the update. For real parameters, $\zeta_{j,t}$ is real and $\Psi_{j,t}$ vanishes, so AURA reduces to a Dice-like test of sign and magnitude; whether the gains carry over to real-valued neural networks is an open question. Convolutional networks were excluded to delimit the scope of this work and are the natural next test. Finally, for clear computational cost reasons, we tested the proposed method on relatively small networks; whether the results extend to networks with billions of parameters, where the memory overhead matters most, remains to be assessed.

Data and software availability

The code that reproduces all numerical experiments of Section IV, including the scripts that regenerate the tables and figures, is available at https://github.com/enricoballini/aura_optimizer_tests.git. The setup has been tested on Ubuntu 24.04.

AURA is distributed as the open-source Python package `aura-optax` (MIT license), which can be installed with `pip install aura-optax`; it requires Python 3.10 or later and depends only on JAX and Optax.

The package provides `adam_aura` and `muon_aura`, which implement Adam-AURA and Muon-AURA, and `scale_by_aura`, which implements the AURA step alone and can be chained after any other update rule.
For complex-valued parameters, the gradients returned by `jax.grad` must be conjugated before being passed to `update`, as for every Optax optimizer.

Acknowledgment

This work was supported by the Danish Research Council for Independent Research through the grant no. 2035-00142B "Network-inspired models to predict the strength of heterogeneous materials". The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] L. Bottou, F. E. Curtis, and J. Nocedal, "Optimization methods for large-scale machine learning," SIAM Review, vol. 60, no. 2, pp. 223–311.

[2] J. Zhuang, T. Tang, Y. Ding, S. C. Tatikonda, N. Dvornek, X. Papademetris, and J. Duncan, "AdaBelief optimizer: Adapting stepsizes by the belief in observed gradients," in Advances in Neural Information Processing Systems, H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, Eds., vol. 33. Curran Associates, Inc., 2020, pp. 18 795–18 806.

[3] S. K. Roy, M. E. Paoletti, J. M. Haut, S. R. Dubey, P. Kar, A. Plaza, and B. B. Chaudhuri, "AngularGrad: A new optimization technique for angular convergence of convolutional neural networks," arXiv:2105.10190, 2021.

[4] S. R. Dubey, S. Chakraborty, S. K. Roy, S. Mukherjee, S. K. Singh, and B. B. Chaudhuri, "diffGrad: An optimization method for convolutional neural networks," IEEE Transactions on Neural Networks and Learning Systems, vol. 31, no. 11, pp. 4500–4511, 2020.

[5] J. Lu, AdaSmooth: An Adaptive Learning Rate Method Based on Effective Ratio. Springer Nature Singapore, 2023, pp. 273–293.

[6] A. G. Baydin, R. Cornish, D. M. Rubio, M. Schmidt, and F. Wood, "Online learning rate adaptation with hypergradient descent," in International Conference on Learning Representations, 2018. [Online]. Available: https://openreview.net/forum?id=BkrsAzWAb

[7] S. L. Goh and D. Mandic, "A class of gradient-adaptive step size algorithms for complex-valued nonlinear neural adaptive filters," in Proceedings. (ICASSP '05). IEEE International Conference on Acoustics, Speech, and Signal Processing, 2005., vol. 5. IEEE, 2005, pp. 253–256.

[8] S. L. Goh and D. P. Mandic, "Stochastic gradient-adaptive complex-valued nonlinear neural adaptive filters with a gradient-adaptive step size," IEEE Transactions on Neural Networks, vol. 18, no. 5, pp. 1511–1516, 2007.

[9] J. Duchi, E. Hazan, and Y. Singer, "Adaptive subgradient methods for online learning and stochastic optimization," Journal of Machine Learning Research, vol. 12, no. 61, pp. 2121–2159, 2011. [Online]. Available: http://jmlr.org/papers/v12/duchi11a.html

[10] M. D. Zeiler, "ADADELTA: An adaptive learning rate method," arXiv:1212.5701, 2012.

[11] M. Eckhoff and M. Reiher, "Lifelong machine learning potentials," Journal of Chemical Theory and Computation, vol. 19, no. 12, pp. 3509–3525, 2023.

[12] ——, "CoRe optimizer: an all-in-one solution for machine learning," Machine Learning: Science and Technology, vol. 5, no. 1, p. 015018, 2024.

[13] F. Schneider, L. Balles, and P. Hennig, "DeepOBS: A deep learning optimizer benchmark suite," in International Conference on Learning Representations, 2019. [Online]. Available: https://openreview.net/forum?id=rJg6ssC5Y7

[14] R. M. Schmidt, F. Schneider, and P. Hennig, "Descending through a crowded valley — benchmarking deep learning optimizers," 2021. [Online]. Available: https://openreview.net/forum?id=k2Om84I9JuX

[15] D. Choi, C. J. Shallue, Z. Nado, J. Lee, C. J. Maddison, and G. E. Dahl, "On empirical comparisons of optimizers for deep learning," 2020. [Online]. Available: https://openreview.net/forum?id=HygrAR4tPS

[16] E. Kiyani, K. Shukla, J. F. Urbán, J. Darbon, and G. E. Karniadakis, "Optimizing the optimizer for physics-informed neural networks and kolmogorov-arnold networks," Computer Methods in Applied Mechanics and Engineering, vol. 446, p. 118308, 2025.

[17] K. S. Mayer, J. A. Soares, A. A. Cruz, and D. S. Arantes, "Adaptive learning rate methods for complex-valued neural networks," IEEE Transactions on Neural Networks and Learning Systems, vol. 36, no. 12, pp. 20 157–20 170, 2025.

[18] M. Rutkowski and P. A. Kowalski, "The impact of optimiser choice on the training dynamics of complex-valued neural networks: A comparative study with real-valued counterparts," International Journal of Approximate Reasoning, vol. 197, p. 109722, 2026.

[19] C. Trabelsi, O. Bilaniuk, Y. Zhang, D. Serdyuk, S. Subramanian, J. F. Santos, S. Mehri, N. Rostamzadeh, Y. Bengio, and C. J. Pal, "Deep complex networks," in International Conference on Learning Representations, 2018. [Online]. Available: https://openreview.net/forum?id=H1T2hmZAb

[20] C. Lee, H. Hasegawa, and S. Gao, "Complex-valued neural networks: A comprehensive survey," IEEE/CAA Journal of Automatica Sinica, vol. 9, no. 8, pp. 1406–1426, 2022.

[21] E. Cole, J. Cheng, J. Pauly, and S. Vasanawala, "Analysis of deep complex-valued convolutional neural networks for mri reconstruction and phase-focused applications," Magnetic Resonance in Medicine, vol. 86, no. 2, pp. 1093–1109, 2021.

[22] M. K. Almansoori and M. Telek, "Performance evaluation of complex-valued neural networks on real and complex-valued classification and reconstruction tasks," Machine Learning with Applications, vol. 22, p. 100742, 2025.

[23] R. Savitha, S. Suresh, and N. Sundararajan, "Metacognitive learning in a fully complex-valued radial basis function neural network," Neural Computation, vol. 24, no. 5, pp. 1297–1328, 2012.

[24] B. Zhang, Y. Liu, J. Cao, S. Wu, and J. Wang, "Fully complex conjugate gradient-based neural networks using wirtinger calculus framework: Deterministic convergence and its application," Neural Networks, vol. 115, pp. 50–64, Jul. 2019.

[25] Z. Dong and H. Huang, "A training algorithm with selectable search direction for complex-valued feedforward neural networks," Neural Networks, vol. 137, pp. 75–84, 2021.

[26] Y. Zhang, Q. Hua, D. Xu, H. Li, and H. Mu, "A complex-valued convolutional neural network with different activation functions in polarimetric sar image classification," in 2019 International Radar Conference (RADAR). IEEE, 2019, pp. 1–4.

[27] G. Hinton, N. Srivastava, and K. Swersky, "Neural networks for machine learning, lecture 6e: rmsprop: Divide the gradient by a running average of its recent magnitude," 2012, lecture slides. [Online]. Available: https://www.cs.toronto.edu/~tijmen/csc321/slides/lecture_slides_lec6.pdf

[28] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," arXiv:1412.6980, 2014.

[29] S. J. Reddi, S. Kale, and S. Kumar, "On the convergence of Adam and beyond," in International Conference on Learning Representations, 2018.

[30] Q. Tong, G. Liang, and J. Bi, "Calibrating the adaptive learning rate to improve convergence of adam," Neurocomputing, vol. 481, pp. 333–356, 2022.

[31] T. Dozat, "Incorporating Nesterov momentum into Adam," in International Conference on Learning Representations (ICLR), Workshop Track, 2016. [Online]. Available: https://openreview.net/pdf?id=OM0jvwB8jIp57ZJjtNEZ

[32] H. Zhang and D. P. Mandic, "Is a complex-valued stepsize advantageous in complex-valued gradient learning algorithms?" IEEE Transactions on Neural Networks and Learning Systems, vol. 27, no. 12, pp. 2730–2735, 2016.

[33] Y. Zhang and H. Huang, "Adaptive complex-valued stepsize based fast learning of complex-valued neural networks," Neural Networks, vol. 124, pp. 233–242, 2020.

[34] W. Zhao and H. Huang, "Adaptive orthogonal gradient descent algorithm for fully complex-valued neural networks," Neurocomputing, vol. 546, p. 126358, Aug. 2023.

[35] X. Yin and H. Huang, "Flatness guided adaptive gradient descent algorithm for fully complex-valued neural networks," in 2026 38th Chinese Control and Decision Conference (CCDC). IEEE, 2026, pp. 5944–5949.

[36] K. Zhang, H. Zhang, and X. Wang, "A hybrid complex spectral conjugate gradient learning algorithm for complex-valued data processing," Engineering Applications of Artificial Intelligence, vol. 133, p. 108352, Jul. 2024.

[37] K. Zhang and H. Zhang, "An adaptive complex-valued stepsize training scheme for complex-valued neural networks," Journal of Applied Mathematics and Computing, vol. 72, no. 1, 2025.

[38] W. Zhao and H. Huang, "Adaptive stepsize estimation based accelerated gradient descent algorithm for fully complex-valued neural networks," Expert Systems with Applications, vol. 236, p. 121166, 2024.

[39] Y. Zhang, H. Huang, and G. Shen, "Adaptive cl-bfgs algorithms for complex-valued neural networks," IEEE Transactions on Neural Networks and Learning Systems, vol. 34, no. 9, pp. 6313–6327, 2023.

[40] Y. Wang, Z. Wang, and H. Huang, "Stochastic adaptive cl-bfgs algorithms for fully complex-valued dendritic neuron model," Knowledge-Based Systems, vol. 277, p. 110788, 2023.

[41] K. Jordan, Y. Jin, V. Boza, Y. Jiacheng, F. Cesista, L. Newhouse, and J. Bernstein, "Muon: An optimizer for hidden layers in neural networks," 2024. [Online]. Available: https://kellerjordan.github.io/posts/muon/

[42] L. Egghe and L. Leydesdorff, "The relation between pearson's correlation coefficient r and salton's cosine measure," Journal of the American Society for Information Science and Technology, vol. 60, no. 5, pp. 1027–1036, 2009.

[43] R. A. Jacobs, "Increased rates of convergence through learning rate adaptation," Neural Networks, vol. 1, no. 4, pp. 295–307, 1988.

[44] M. Riedmiller and H. Braun, "A direct adaptive method for faster backpropagation learning: the rprop algorithm," in IEEE International Conference on Neural Networks. IEEE, pp. 586–591.

[45] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in International Conference on Learning Representations, 2019. [Online]. Available: https://openreview.net/forum?id=Bkg6RiCqY7

[46] L. Balles and P. Hennig, "Dissecting adam: The sign, magnitude and variance of stochastic gradients," in Proceedings of the 35th International Conference on Machine Learning, ser. Proceedings of Machine Learning Research, J. Dy and A. Krause, Eds., vol. 80. PMLR, 10–15 Jul 2018, pp. 404–413. [Online]. Available: https://proceedings.mlr.press/v80/balles18a.html

[47] H. Kesten, "Accelerated stochastic approximation," The Annals of Mathematical Statistics, vol. 29, no. 1, pp. 41–59, 1958.

[48] G. Saridis, "Learning applied to successive approximation algorithms," IEEE Transactions on Systems Science and Cybernetics, vol. 6, no. 2, pp. 97–103, 1970.

[49] A. G. Barto and R. S. Sutton, "Goal seeking components for adaptive intelligence: An initial assessment," Air Force Wright Aeronautical Laboratories, Avionics Laboratory, Wright-Patterson Air Force Base, Ohio, Tech. Rep. AFWAL-TR-81-1070, Apr. 1981, technical Report.

[50] M. Riedmiller, "Advanced supervised learning in multi-layer perceptrons — from backpropagation to adaptive learning algorithms," Computer Standards & Interfaces, vol. 16, no. 3, pp. 265–278, 1994.

[51] A. Kantsila, M. Lehtokangas, and J. Saarinen, "Complex rprop-algorithm for neural network equalization of gsm data bursts," Neurocomputing, vol. 61, pp. 339–360, 2004.

[52] C.-A. Popa, "Enhanced gradient descent algorithms for complex-valued neural networks," in 2014 16th International Symposium on Symbolic and Numeric Algorithms for Scientific Computing. IEEE, 2014, pp. 272–279.

[53] J. Liu, C. Lin, C. Li, L. Sheng, M. Sun, J. Yan, and W. Ouyang, "Adaptive gradient method with resilience and momentum," arXiv:2010.11041, 2020.

[54] P. Malviya, G. Mordido, A. Baratin, R. B. Harikandeh, G. K. Dziugaite, R. Pascanu, and S. Chandar, "Torque-aware momentum," arXiv:2412.18790, 2024.

[55] N. McGreivy and A. Hakim, "Weak baselines and reporting biases lead to overoptimism in machine learning for fluid-related partial differential equations," Nature Machine Intelligence, vol. 6, no. 10, pp. 1256–1269, Sep. 2024.

[56] X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," Journal of Machine Learning Research, vol. 9, pp. 249–256, 2010.

[57] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, Y. Katariya, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang, "JAX: composable transformations of Python+NumPy programs," 2018, open-source software.

[58] M. Calafà, E. Hovad, A. P. Engsig-Karup, and T. Andriollo, "Physics-informed holomorphic neural networks (PIHNNs): Solving 2D linear elasticity problems," Computer Methods in Applied Mechanics and Engineering, vol. 432, p. 117406, Dec. 2024.

[59] E. Ballini, A. P. Engsig-Karup, and T. Andriollo, "A holomorphic neural network framework for 3d boundary value problems governed by harmonic potentials," Computer Methods in Applied Mechanics and Engineering, Accepted, arXiv:2605.31231, 2026.

[60] R. Savitha, S. Suresh, N. Sundararajan, and P. Saratchandran, "A new learning algorithm with logarithmic performance index for complex-valued neural networks," Neurocomputing, vol. 72, no. 16-18, pp. 3771–3781, 2009.

[61] R. Savitha, S. Suresh, and N. Sundararajan, "A self-regulated learning in fully complex-valued radial basis function networks," in The 2010 International Joint Conference on Neural Networks (IJCNN). IEEE, 2010, pp. 1–8.

[62] M. F. Amin, R. Savitha, M. I. Amin, and K. Murase, "Complex-valued functional link network design by orthogonal least squares method for function approximation problems," in The 2011 International Joint Conference on Neural Networks. IEEE, 2011, pp. 1489–1496.

[63] R. Savitha, S. Suresh, and N. Sundararajan, "A meta-cognitive learning algorithm for a fully complex-valued relaxation network," Neural Networks, vol. 32, pp. 209–218, 2012.

[64] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, "Algorithms for hyper-parameter optimization," in Advances in Neural Information Processing Systems, J. Shawe-Taylor, R. Zemel, P. Bartlett, F. Pereira, and K. Weinberger, Eds., vol. 24. Curran Associates, Inc., 2011. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2011/file/86e8f7ab32cfd12577bc2619bc635690-Paper.pdf

[65] Y. Ozaki, S. Watanabe, and T. Yanase, "Optunahub: A platform for black-box optimization," Journal of Machine Learning Research, vol. 27, no. 203, pp. 1–10, 2026. [Online]. Available: http://jmlr.org/papers/v27/25-2424.html

APPENDIX
SALTON-LIKE VERSUS DICE-LIKE MEASURE

The numerator of $\zeta_{j,t}$ in (2b) may be normalized either by the geometric mean of $|d_{j,t}|^2$ and $|d_{j,t-1}|^2$, that is, $|d_{j,t}||d_{j,t-1}|$, which gives the Salton-like measure $\zeta_{j,t}^{\mathrm{Sal}}$, or by their arithmetic mean, which gives the Dice-like measure used here. Their real parts are Salton's cosine and Dice's measure of the associated $\mathbb{R}^2$ vectors; their imaginary parts have no counterpart in those

measures, hence the suffix "-like". Writing $r_{j,t} = |d_{j,t}|/|d_{j,t-1}|$ for the ratio of consecutive step lengths, the two differ by

$$\frac{|\zeta_{j,t}|}{|\zeta_{j,t}^{\mathrm{Sal}}|} = \frac{2|d_{j,t}||d_{j,t-1}|}{|d_{j,t}|^2 + |d_{j,t-1}|^2} = \frac{2r_{j,t}}{1 + r_{j,t}^2} \in (0, 1], \tag{19}$$

which equals 1 only for $r_{j,t} = 1$ and decays as the two lengths separate, see Figure 6 and [42]. The Dice-like measure therefore damps $\zeta_{j,t}$ whenever consecutive directions differ markedly in magnitude, which is the desired behavior, as it highlights updates that differ in magnitude, and is used for this reason.

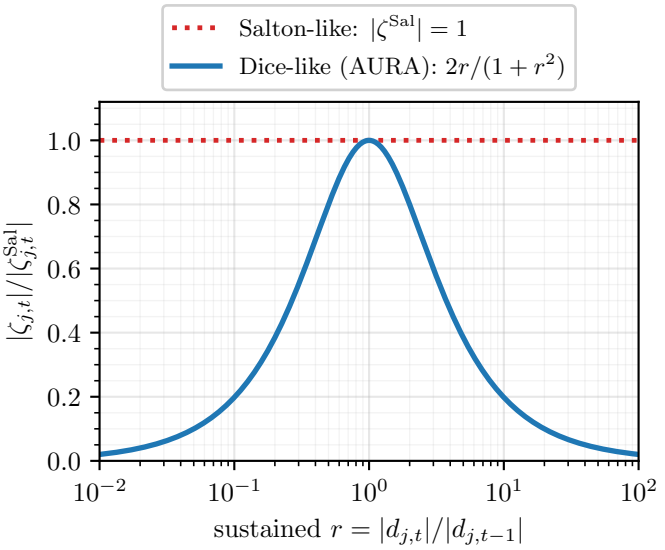


Fig. 6. Dice-like against Salton-like measure as a function of the ratio $r = |d_{j,t}|/|d_{j,t-1}|$ of consecutive step lengths, see (19). The Salton-like measure has unit modulus for every r, whereas the Dice-like one decays as the two lengths separate.